

Phoenix Evolution Patch - Manual y base de conocimiento (PES 2021)

Auditoría de parches - generado 2026-10-09 04:16

Manual del Parche Phoenix Evolution (PES 2021)

Versión del manual: 2026-10-09 (madrugada, Lima) - **Autor del proyecto:** FRALEX - Redactado con Claude.

Para qué sirve: reúne **todo lo aprendido y probado** para construir el **Phoenix Evolution Patch**: qué es cada pieza, cómo funciona el juego por dentro, las recetas paso a paso, los errores que no hay que repetir y la ruta para fabricarlo.

Regla de lectura: si algo de aquí choca con un documento más viejo, **manda este manual**. Lo más nuevo y detallado está en `base-conocimiento/05-PUERTA-EN-VIVO.md` y en el diario `PRUEBAS.md`.

Leyenda: - [PROBADO] **probado en el juego**; - [VERDE] **se puede**: mismo método ya probado, falta hacerlo; - [AMARILLO] **con investigación**: el camino es conocido; - [ROJO] **frontera**: nadie lo ha hecho; - [NO] **no funciona**.

1. La visión

El **Phoenix Evolution Patch** quiere ser el parche de PES 2021 **más completo, estable y conectado** que exista: un solo sistema con todo el ecosistema de FRALEX.

Pieza	Qué es	Dónde vive
Web Phoenix Evolution	Liga (Galaxy League), mercado de fichajes, perfiles, buzón de avisos	repo Fralexito/phoenixevolution (rama borrador) + Supabase
Phoenix Link	App de PC: salas para jugar a distancia por Parsec (fork de Smash Soda 7.0.3) y cartero web <-> juego	repo Fralexito/smash-soda-fork, rama rediseño-phoenix-portal, carpeta SmashSoda/
Phoenix Sync (antes «Phoenix Mercado»)	Programa que mete en el juego lo que dice la web: option file, Liga Máster y base de datos	mismo repo, rama mercado-fase0, carpeta PhoenixSync/
Módulo Sider phoenix.lua	Código Phoenix dentro del juego: muestra avisos y espía lecturas	PhoenixSync/sider/phoenix.lua
El parche	Base de datos + contenido + sider.ini limpio + lanzador	por construir (ver §7)

El camino de los datos:

```
WEB (Supabase) <-> PHOENIX LINK (cartero, internet) <-> archivos en la PC <-> SIDER (dentro del juego) <-> PES 2021
                                     ^
                                     PHOENIX SYNC (fabrica los archivos: option file, Liga Máster, base de datos)
```

El juego nunca toca internet: todo lo online pasa por Phoenix Link. Sider no tiene red, y eso lo hace más seguro.

2. Cómo funciona PES 2021 por dentro (lo esencial)

2.1 Capas del juego

- **El ejecutable** PES2021.exe (1.07.02 + Data Pack 7.0). Las reglas «cableadas» (tamaños de liga, IA, físicas) solo se cambian parcheando código. [ROJO]
- **Paquetes CPK** listados en `DpFileList.bin` (carpeta download). Es el modelo clásico; lo usa Sudamerican.

Sider (`SiderAddons\sider.exe`, `sider.dll`, `sider.ini`). Entrega archivos sueltos al juego (**livecpk**) y corre módulos Lua. Lo usa ConneGOL para casi todo.

- **Gana la primera raíz** `cpk.root` que tenga el archivo pedido.
- **Carpeta save:** `EDIT00000000` (option file, cifrado), `ML0000000N` (Liga Máster, ranura N = archivo N-1 en hexadecimal) y `SYSTEM00000000`.

2.2 La base de datos (`common\etc\pesdb\*.bin`)

Cada archivo va comprimido con cabecera **WESYS**: 16 bytes = `ff 10 81 57` + "WESYS" + tamaño comprimido u32 + tamaño real u32, seguidos del contenido en **zlib**.

- **El juego acepta una recompresión distinta** (zlib nivel 9). [PROBADO]

- **Archivos principales:** Player.bin (fichas de 312 B, ID u32 en +8), PlayerAssignment.bin (16 B: jugador en +4, equipo en +8), Team.bin (1532 B, ID en +8), Competition.bin (36 B), CompetitionEntry.bin (12 B), Coach.bin, Stadium.bin, Country.bin, Tactics*.bin, PlayerWeekly.bin y TeamWeekly.bin (forma semanal), Boots.bin, Glove.bin, Derby.bin...
- **Límites del juego:** 750 equipos, 30.000 jugadores, 40 por plantilla. No cambiar el tamaño de una liga sin parchear el exe, porque cuelga la Liga Máster.

2.3 Cuándo lee el juego la base ([PROBADO] medido con el espía livecpk_read, 2026-10-09)

Momento	¿Relee pesdb?
Al arrancar	Sí, todo, dos veces (arranque y «Datos del sistema: Cargando»)
Al entrar a Editar	Sí (Player, PlayerAssignment, Team, Coach, Competition...), pero en un borrador que se descarta al salir
Editar -> Cargar	Sí, y lo aplica al juego [PROBADO]
Al entrar a la Liga Máster	Solo InstallVersionPlayer.bin
Al abrir pantallas de jugadores, armar un amistoso o cambiar «Datos Actual. en vivo»	No

2.4 Mapa de Player.bin (registro de 312 B; bits desde el inicio; 6 bits = valor - 40)

19 habilidades de campo [PROBADO]:

- ataque 370, control 281, regate 352, posesión estrecha 416;
- pase raso 263, pase bombeado 402, finalización 396, cabezazo 288;
- balón parado 250, efecto 332;
- velocidad 306, aceleración 344, equilibrio 376, contacto físico 390, salto 294, resistencia 338;
- defensa 275, recuperación 312, agresividad 384.
- **Potencia de tiro** 358 [PROBADO] (calibrada con la pantalla del juego).

Físico y datos [PROBADO]:

- altura 216 (8 bits, +100);
- peso 256 (7 bits, +30);
- edad 408 (6 bits, +15);
- posición principal 434 (4 bits);
- nacionalidad 233 (9 bits -> Country.bin).
- **Portero:** bits 269, 300, 320, 326, 364 [OBSERVADO]. Falta saber cuál es cuál: hace falta una foto de la ficha de Joan García.

Posiciones jugables (2 bits: 0 no, 1 B, 2 A) [OBSERVADO]:

- PT 350, DFC 468, LI 318, LD 474;
- MCD 414, MC 456, MI 466, MD 460, MO 464;
- EI 472, ED 476, SD 478, DC 470.
- **Pie malo** uso 454 / precisión 462, **forma** 438, **lesiones** 458 [HIPÓTESIS]. El juego **varía** los bytes 54-57 al cargar.
- **Habilidades especiales y estilos COM:** bits 480-531 [HIPÓTESIS].
- **Nombres** dentro de la ficha (en mayúsculas en el byte 129 y otra vez en el 190; en formato normal en el 251).

Imágenes: - minicaras: livecpk\Logos\common\render\symbol\player\<ID>.dds; - escudos: ...symbol\flag\e_<6 dígitos>_l.png; - caras 3D: carpeta face\real\<ID>\.

2.5 La base viva de ConmeGOL

El juego **no** usa CGP_database.cpk. Usa la base que sirve Sider desde SiderAddons\olmosjr23\Database (5/9/2026). El catálogo de la web hay que regenerarlo desde ahí.

3. Las piezas que YA funcionan

Capacidad	Estado	Cómo
Mover jugadores en el option file (alineación y dorsales correctos)	[PROBADO]	PhoenixSync.exe mover / sincronizar

Liga Máster: vender, traspasos de la IA, fichar para el usuario, presupuesto y tope salarial	[PROBADO]	Phoenix Sync, con el juego en el menú (el juego relee el guardado al cargar la ranura)
Cambios firmados desde la web (Ed25519), aplicados todo o nada	[PROBADO]	/liga/cambios
Avisos web -> juego en vivo (1-2 s)	[PROBADO]	Web -> Phoenix Link -> content\phoenix\avisos.txt -> phoenix.lua -> overlay de Sider
Base de datos propia servida por Sider (stats cambiadas al arrancar)	[PROBADO]	Raíz livecpk\Phoenix-DB antes de olmosjr23\Database
Cambiar stats sin reiniciar	[PROBADO]	Archivo nuevo + Editar -> Cargar
Leer la memoria del juego sin riesgo	[PROBADO]	ReadProcessMemory sobre el propio proceso (copia segura)
Página «Vestuario» con la plantilla editable	[PROBADO] prototipo	artifact con base de datos; los cambios quedan «pendientes»
295/295 pruebas automáticas de Phoenix Sync	[PROBADO]	PhoenixSyncPruebas.exe

4. Recetas paso a paso

R1 - Cambiar stats de un jugador en vivo [PROBADO]

- **Phoenix Sync** toma la base que sirve el parche (olmosjr23\...\Player.bin) y la descomprime.
- Busca la ficha por ID (u32 en +8) y cambia **solo** los bits del campo, conservando los demás bits del byte.
- Recomprime con WESYS + zlib. Verifica de ida y vuelta (descomprimir y comparar).
- Escribe en SiderAddons\livecpk\Phoenix-DB\common\etc\pesdb\Player.bin de forma **atómica** (.tmp y luego renombrar), en la raíz del juego y en la carpeta del modo.
- En el juego: **Editar -> Cargar** (sin guardar).
- El siguiente partido o pantalla nueva muestra el cambio.

Comprobación: el espía de phoenix.lua anota cada lectura de player.bin con una **huella** (suma cada 64 bytes del trozo leído). Compárala con la del archivo.

R2 - Mandar un aviso al juego [PROBADO]

- **Desde la web:** tarjeta «Mandar aviso a mi juego». Phoenix Link revisa cada 15 s con el juego abierto y escribe avisos.txt.
- **A mano:** escribir SiderAddons\content\phoenix\avisos.txt (UTF-8 sin BOM, máx. 14 líneas, de forma atómica).
- **En el juego:** barra espaciadora -> tecla 1 hasta «PHOENIX EVOLUTION».

R3 - Instalar algo en un parche con «switcher» [PROBADO]

- El switcher de ConmeGOL (PES2021 Start.exe, Python con PyInstaller) hace robocopy "ConmeGol Extras\<modo>" "<juego>" /E /IS /IT, escribe version_actual.txt, abre sider.exe y, 2 s después, PES2021.exe.
- **Por eso todo va en ConmeGol Extras\<modo>\SiderAddons;** si no, se pierde. Lo que se cambia **en vivo** se escribe en la raíz del juego.

R4 - Probar un módulo Sider sin romper nada [PROBADO]

- **Sintaxis:** luajit -bl modulo.lua.
- **Simulación con los globales exactos de Sider 7.3.3:** assert, pairs, ipairs, tostring, tonumber, type, error, unpack, collectgarbage, table, string, math, os, io, fs, ffi, jit, bit, zlib, memory, match, audio, input, log. **No hay pcall.**
- Memoria y ffi falsos para probar búsquedas y escrituras, incluida una zona que «desaparece» a mitad de la prueba.
- **Instalar con respaldo** de sider.ini y **una sola línea** nueva. Comprobar con diff.
- **Ver sider.log:** «Module (x) is NOT activated» explica cualquier fallo de arranque.

R5 - Día de actualización de un parche (ConmeGOL / Sudamerican)

- **Antes:** copia de la carpeta save.
- **Comparar con la foto del 8/10** (_PhoenixMercado_prueba_versiones\2026-10-08\): base, DpFileList, Sider y

option file.

- **Regenerar livecpk\Phoenix-DB** desde la base nueva. **Si no, tapa los datos nuevos del parche.**
- **Auditar** solo los módulos Sider nuevos o cambiados.
- **Regenerar el catálogo** y subirlo a la web.
- **Probar** y anotar en PRUEBAS.md, parches/<parche>.md y Drive.

R6 - Operaciones de Liga Máster [PROBADO]

- Siempre con el juego **en el menú** (la ranura sin cargar) y con copia de seguridad.
- El juego relee el guardado al entrar a la ranura.
- Detalle técnico: liga-master/ESTRUCTURA-ML.md y base-conocimiento/01-MOTOR-PES2021.md.

5. Reglas de oro técnicas (aprendidas a golpes)

- **Nunca leer ni escribir memoria del juego directo.** Otros hilos (ReShade, DLSS) la liberan en cualquier momento: crash 0xC0000005 comprobado. Hay que usar ReadProcessMemory / WriteProcessMemory, que fallan sin cerrar el juego.
- **Mejor archivo que memoria.** El juego rehace sus copias desde la base, así que escribir en memoria no sirve para las stats. El camino es **archivo + Editar -> Cargar**.
- **El Lua de Sider no trae pcall.** Un error en init deja el módulo apagado. Para contar errores se usa una «bandera».
- **ffi.cdef no deja redeclarar.** Con Shift+R el Lua es el mismo, así que se usan nombres con versión (phx06_* + __asm__("FunciónReal")).
- **Shift+R solo recarga módulos que ya estaban activos.**
- **Buscar fichas por ID y verificar sus cualidades.** El juego varía bits (forma, lesión), así que una ficha nunca coincide byte a byte.
- **El switcher pisa la raíz.** Lo permanente va en la carpeta del modo.
- **La actualización en vivo de Konami ya no existe:** sus servidores cerraron el 25/08/2022.
- **Respalos, siempre.** Un cambio = una línea en REGISTRO.md. Cada prueba en el juego se anota en PRUEBAS.md, salga bien o mal.
- **0 errores:** si algo falla, no se avanza hasta entenderlo y arreglarlo. «Si algo se rompe, mejor no lo hagas.»
- **Contenido de terceros** (caras, kits, estadios, módulos) solo con **permiso y crédito**. Lo propio (base, programas, conexión) es de FRALEX. **Nunca se distribuye el juego.**

6. El universo que abrió la «puerta en vivo»

Qué	Archivo	Estado
Stats y habilidades	Player.bin	[PROBADO]
Fichajes y traspasos en vivo	PlayerAssignment.bin	[VERDE] (siguiente experimento)
Entrenadores	Coach.bin	[VERDE]
Equipos	Team.bin	[VERDE]
Ascensos, descensos, participantes de cada liga	CompetitionEntry.bin	[VERDE] (ojo: los módulos CGP comprueban equipos por competición)
Estadios, botines y guantes	Stadium.bin, Boots.bin, Glove.bin	[VERDE]
Forma semanal propia	PlayerWeekly.bin / TeamWeekly.bin	[AMARILLO]
Habilidades en la Liga Máster	Player.bin (la LM no las guarda aparte)	[VERDE] por confirmar en una carrera

Productos posibles: 1. **Mercado real conectado:** un fichaje en la web o en la vida real aparece en el juego. 2. **Phoenix Weekly:** stats según el rendimiento real, con reglas fijas y aprobación de FRALEX. **Sin copiar notas de otros juegos.** 3. **Liga online justa:** todos con el mismo archivo, verificado por huella SHA-256. 4. **Eventos:** jugador de la semana, lesiones o suspensiones, homenajes. 5. **Liga Máster conectada** a la web.

7. Ruta para construir el parche (actualizada)

Fase	Qué	Estado
0. Terreno	Instalación limpia de referencia (¿PES 2021 de Steam?), exe 1.07.02 + DP 7.0, Sider fijado, partido de referencia	[PENDIENTE]
1. Núcleo	Base de datos Phoenix (Liga 1 + Liga 2 + Copa; lo mejor de ConmeGOL y Sudamerican con permiso), option file oficial de la web, sider.ini limpio y auditado	[PENDIENTE] (la técnica de base propia ya está [PROBADO])
2. Herramientas	Phoenix Sync fabrica Player/PlayerAssignment/... (WESYS), CPK y DpFileList; Phoenix Doctor (auditoría automática de Sider); Auditor de parches; lanzador sin .bat	[VERDE] en parte
3. Conexión	Avisos [PROBADO] -> cambios de stats en vivo [PROBADO] -> fichajes en vivo [VERDE] -> resultado automático (match.stats() -> resultado.json -> Link -> web) [VERDE] -> verificador del anfitrión [VERDE]	En marcha
4. Contenido	Addons por liga (caras, kits, estadios) con permisos y créditos; menos archivos sueltos; texturas bien comprimidas	[PENDIENTE]
5. Más allá	Competiciones propias [AMARILLO], estadísticas completas [AMARILLO], Liga Máster online asíncrona [ROJO] (la idea más original), crear jugadores [AMARILLO], avisos con aspecto nativo [AMARILLO]	Futuro

Inmediato (para la próxima sesión): 1. Fichaje en vivo con PlayerAssignment.bin + Editar -> Cargar. 2. Confirmar si los cambios de stats llegan a la Liga Máster. 3. Automatizar: Vestuario web -> Phoenix Sync arma el archivo -> Phoenix Link lo coloca -> aviso «Editar -> Cargar». Los prompts para LINK y WEB están por escribir. 4. Calibrar el portero, el pie malo, la forma y las lesiones con fotos de la ficha de Joan García. 5. Actualización de ConmeGOL: aplicar la receta R5 y regenerar Phoenix-DB.

8. Futuro (deseos de FRALEX, guardados para no olvidarlos)

Crear jugadores que no existen (juveniles, jugadores reales que faltan, jugadores de la liga).

- **Hipótesis:** fichas nuevas en Player.bin (ID libre) + su fila en PlayerAssignment.bin.
- **Falta investigar:** rango de IDs seguros, PlayerDeleteList.bin, InstallVersionPlayer.bin y el efecto en la Liga Máster.
- **Hacer faces:** FRALEX quiere aprender. Claude le dará el **ID exacto** de cada jugador y **la carpeta** (face\real\<ID>\, minicara symbol\player\<ID>.dds), revisará los nombres de archivo y lo instalará en una raíz Phoenix sin pisar las del parche.
- **Avisos con aspecto nativo:** imágenes de los menús generadas por Phoenix Link y servidas por Sider, en lugar del overlay.
- **Botones nativos en vez de Sider:** el overlay es solo de laboratorio. En la versión final todo se aplica solo, y el jugador usa la web, Phoenix Link y un único botón del juego (Editar -> Cargar).
- **PES online propio:** Phoenix Link (Parsec) + verificador + resultado automático. Un servidor propio estilo PESBUL es [ROJO] (meses o años).
- **Liga Máster online asíncrona [ROJO]:** cada amigo con su club y su carrera, compartiendo mercado y tabla en la web.

9. Dónde está todo

Qué	Dónde
Este manual	PhoenixSync/MANUAL-PARCHE-PHOENIX.md (y en Drive)
Base de conocimiento	PhoenixSync/base-conocimiento/ (00-05, parches/, investigacion/, datos/) + PDF base-conocimiento-PES21.pdf
Diario de pruebas en el juego	PhoenixSync/PRUEBAS.md
Bitácora de cambios	PhoenixSync/REGISTRO.md (Sync) - REGISTRO-LINK.md (Link, en su rama) - REGISTRO.md del repo web
Sider	PhoenixSync/sider/ (phoenix.lua, VINCULO-TIEMPO-REAL.md, RIESGOS-SIDER.md, AUDITORIA-SIDER.md)
Prompts para los otros chats	PhoenixSync/prompts/ - traspaso completo: PhoenixSync/PROMPT-PARCHE-PHOENIX.md
Coordinación entre chats y cuentas	COORDINACION.md, CLAUDE.md
Juego ConmeGOL	D:\Frank\Games_\Conmegol Patch\ (Sider en SiderAddons\, modos en ConmeGol Extras\)
Juego Sudamerican	D:\SP2026\ ...\Conmegol Patch_PhoenixMercado_prueba\ (exe de Sync, .bat, db\ con las variantes v90/v95/v99, _versiones\2026-10-08\)
Pruebas en la PC	sider.ini.respaldo-phoenix-20261009 y sider.ini.respaldo-phoenixdb-20261009 (en la raíz y en la carpeta del modo)
Respaldos de Sider	
Google Drive	carpeta 15TL096cZpM9A9bLPHtRtpNL6YbczyPul
Página Vestuario Barça	https://claude.ai/artifact/WskFnXgRmMhAs12Mv6cwTh

10. Créditos y respeto

- **ConmeGOL Patch y Sudamerican Project:** referencias de estudio. Su contenido (caras, kits, estadios, módulos) es de sus autores (Olmos Jr 23, facemakers, kitmakers...). Para usarlo en Phoenix: **permiso + crédito**.
- **Herramientas de la comunidad** (Sider de juce, libpesXcrypter, 4ccEditor, pes-file-tools, módulos Lua como CommonLib de zlac): se respetan sus licencias y se citan.
- **Lo propio del Phoenix Evolution Patch** (base de datos, Phoenix Sync, Phoenix Link, el módulo Phoenix y la conexión con la web): **de FRALEX**.

Base de conocimiento - PES 2021 y sus parches

Qué es: todo lo que Phoenix Sync sabe de los archivos de PES 2021, ordenado como una base de datos. Se **alimenta en cada auditoría** y nunca se borra lo aprendido: si algo cambia en una versión nueva del parche, se anota la fecha y el motivo.

Empieza por ../MANUAL-PARCHE-PHOENIX.md: el manual que reúne todo para construir el parche (visión, cómo funciona el juego, recetas, reglas de oro, ruta y futuro).

Cómo está dividida:

Archivo	Qué guarda
01-MOTOR-PES2021.md	Lo que es del juego y vale para cualquier parche: cómo están armados el option file, el guardado de Liga Máster y el bloque comprimido. Tablas con posición, tamaño, campo y estado.
02-PARCHE-PHOENIX.md	El objetivo de fondo: anatomía de un parche de PES 2021, qué ya controlamos y qué falta para fabricar el parche Phoenix vinculado a la web, a Phoenix Link y al Mercado.
03-INVESTIGACION-PARCHES.md	Lo que enseñan los demás parches (FL, Evoweb, ConmeGOL, Sudamerican, Gogosz, PESBUL): causas de crash y la regla Phoenix para cada una, modo online realista, herramientas y formatos para fabricar el parche, y la receta del Phoenix Evolution Patch. Informes completos en investigacion/.
04-ESTRATEGIA-PHOENIX.md	Informe estratégico: ConmeGOL y Sudamerican por dentro (bases de datos comparadas, contenido, Sider), sus estrategias y sus fallos, cómo superarlos en estabilidad y rendimiento, funciones nuevas por viabilidad ([VERDE][AMARILLO][ROJO]), límites del motor y ruta completa por fases.
05-PUERTA-EN-VIVO.md	Cambiar la base sin reiniciar (2026-10-09): Player.bin propio servido por Sider + Editar -> Cargar. Qué falló y por qué, cuándo lee el juego cada archivo, el universo que abre (fichajes, DT, competiciones, forma semanal) y los deseos a futuro (crear jugadores, avisos nativos).
parches/conmegol-26.md	Lo que es solo del ConmeGOL Patch 26: carpetas, archivos, números de equipos y jugadores, IDs, competiciones, rarezas vistas.
parches/sudamerican-2026.md	Lo poco que ya se sabe del Sudamerican Project 2026 (pendiente de auditar).
datos/motor.json	La misma información del motor, en formato de datos, para que el software la lea.
datos/conmegol-26.json	Equipos, IDs y competiciones del ConmeGOL 26 sacados del guardado (627 equipos, 39 competiciones).
../sider/RIESGOS-SIDER.md	Puente en vivo con Sider: qué permite el Lua de Sider (archivos, memoria, overlay), cómo es el puente web -> app -> archivo -> juego, los 12 riesgos con su medida y las fases de prueba.
../sider/AUDITORIA-SIDER.md	Auditoría de los 49 módulos Sider activos: qué toca cada uno, choques encontrados (cámaras que se pisan, escrituras a ciegas), módulos que no hacen nada y plan de limpieza. Detalle en ../sider/auditoria/.
../PRUEBAS.md	Diario de prueba y error: qué se probó en el juego, qué falló y cómo se corrigió.
../liga-master/ESTRUCTURA-ML.md	El cuaderno técnico original (más detalle, menos orden).

Estados que se usan en todas las tablas:

- [PROBADO] **PROBADO**: se cambió en el archivo y se vio el resultado en el juego.
- [OBSERVADO] **OBSERVADO**: se dedujo comparando archivos (antes/después) o contra la pantalla, sin haberlo editado aún.
- [HIPÓTESIS] **HIPÓTESIS**: idea razonable, sin comprobar.
- [NO] **DESCARTADO**: se buscó y no está, o no funciona así.

Regla de oro: nada pasa a [PROBADO] sin una prueba en el juego anotada en PRUEBAS.md.

Cómo auditar un parche nuevo (Sudamerican, próximas versiones de ConmeGOL...): 1. Copia de seguridad de EDIT00000000 y de los ML... de ese parche (carpeta save con su número de Steam). 2. Generar el catálogo con PhoenixSync.exe catalogo (option file + .cpk de la base). 3. Correr los prototipos de liga-master/prototipos/ sobre un guardado descifrado: bloques.py (bloques de alineación), blob.py (bloque comprimido) y la sonda del C++ (tablasDe, alineacionDe). 4. Anotar diferencias respecto al motor en parches/<parche>.md y volcar sus datos a datos/<parche>.json. 5. Repetir las pruebas clave en el juego (vender, traspaso IA<->IA, presupuesto) y anotarlas en PRUEBAS.md.

01 - El motor de PES 2021 (vale para cualquier parche)

Versión del juego: eFootball PES 2021 SEASON UPDATE (PC/Steam). Todo lo de abajo se descubrió sobre el ConmeGOL Patch 26, pero la **forma** de los archivos la pone el juego, no el parche: el parche solo cambia el contenido (jugadores, clubes, competiciones). Lo que dependa del parche está en parches/.

Las posiciones (offsets) se dan en hexadecimal y son las del guardado de referencia. Todo lo que el programa usa de verdad lo localiza **por anclas** (buscando la forma de los datos), no por posición fija.

A. Cifrado y envoltura (option file y guardados)

Qué	Dato	Estado
Cifrado	libpesXcrypter con la clave de PES 2021; se descifra y se vuelve a cifrar idéntico	[PROBADO]
Envoltura	cabecera + descripción (384 B) + logo (17.315 B) + DATOS + serial (44 B)	[PROBADO]
Descripción	bytes 0-127: nombre (el menú Cargar NO lo muestra) - 128-383: «texto info» (lo que SÍ muestra el menú)	[PROBADO]
Hash de cabecera	el juego no lo valida: acepta archivos editados	[PROBADO]
Tamaño de datos variable	el guardado de LM crece/encoge; el C++ re-cifra con cualquier tamaño	[PROBADO]
Ranuras del menú Cargar	ranura N = archivo ML00000000(N-1) en hexadecimal (ranura 11 = ML00000000A)	[PROBADO]
Autoguardado	al terminar un partido el juego sobrescribe la ranura cargada; identificar archivos por huella (md5)	[PROBADO]

B. Option file (EDIT00000000): amistosos, salas y carreras nuevas

Qué	Dónde	Estado
Nº de jugadores editados	u16 en +96	[PROBADO]
Nº de equipos	u16 en +100	[PROBADO]
Jugadores editados	desde 0x7C, 312 B cada uno (mismo formato que Player.bin del CPK)	[PROBADO]
Equipos	desde 0x8ED2FC, 588 B cada uno (nombre en +104, abreviatura en +174)	[PROBADO]
Plantillas	desde 0x9D4648, 284 B por equipo: ID equipo u32 - 40 IDs de jugador u32 - 40 dorsales u16 - 40 B («Unknown A»)	[PROBADO] (mover probado en juego)
Tácticas por equipo	desde 0xA09880, 628 B por equipo, mismo orden que las plantillas: ID equipo u32 - formaciones (varias copias de coordenadas) - orden de formación 40 B en +0x1E4 (índices de plantilla; 0-10 titulares) - 6 roles en +0x20C	[OBSERVADO] (ver §G)

Bits de habilidades (Player.bin / editados)	ataque 370, control 281, regate 352, regate ajustado 416, pase raso 263, pase bombeado 402, finalización 396, cabezazo 288, balón parado 250, efecto 332, velocidad 306, aceleración 344, equilibrio 376, contacto físico 390, salto 294, resistencia 338, defensa 275, recuperación 312, agresividad 384 (6 bits, +40) - altura 216 (8 bits, +100) - peso 256 (7, +30) - edad 408 (6, +15) - posición 434 (4) - nacionalidad 233 (9)	[PROBADO]
---	---	-----------

Corregido (8 oct, 16:30): al mover un jugador el programa actualiza también el bloque de tácticas de los dos equipos, con la misma lógica que la Liga Máster (core/Alineacion.h): el que se va sale del orden y de los roles, los índices mayores bajan uno, si era titular o tenía rol hace falta sustituto (elegido por posición con el catálogo; a un portero solo lo cubre otro portero) y el que llega entra como última reserva. Comprobado en el option file real del ConmeGOL 26: **749 bloques**, todos con el ID del equipo en +0; 711 equipos con el orden como **identidad 0...39** (así lo deja el editor del juego) y 38 equipos vacíos con todo 0xFF. El formato (cola identidad o 0xFF) se conserva. Pendiente la prueba en el juego (amistoso con un option file temporal). Antes de esto, al mover un jugador solo en la plantilla la alineación quedaba corrida (el portero suplente salía de delantero).

C. Guardado de Liga Máster (ML0000000N): estructura general

Zona	Dónde	Qué	Estado
Cabecera	0x00-0x4F (80 B)	10, 0x50, ..., 700, 700; los 32 B de 0x30 cambian en cada guardado y no se validan	[PROBADO]
700 bloques de equipo	0x50 + 1680-k	nombre +4 - ID option +656 - plantilla +0x14C (40 x reg u32, pid u32, relleno 65535,0) - dorsales +0x29A (40 x u16) - contador +0x426 (1 B). k >= 528 = selecciones	[PROBADO]
Alineaciones de la IA	~0x133A30, 629 bloques de 600 B	ver §D	[PROBADO]
Fichas de jugador (596 B)	0x329B00, una por reg (5.759)	pocos campos usados; no llevan el club ni el valor	[OBSERVADO]
Dinero del usuario	0xC7DD00	ver §F	[PROBADO]
Tablas del equipo del usuario	0xBE5FC8 ... 0x1049B3C	ver §E	[PROBADO]
Competiciones	desde ~0xB08000, bloques de 3000 B	lista de equipos de cada competición (IDs u32, relleno 0x0003FFFF)	[OBSERVADO]
Calendario del usuario (meses especiales)	0xA8D6D8 (enero), 0xAAB288 (junio)	1 registro de 708 B por día: fecha - ID competición - rival; en días de liga, 10 IDs de partido u16	[OBSERVADO]
Ranking de clubes (16 B)	0xC016B0, 263 registros	[?][ID interno de club u32][puesto u32][v u16 x2]; Real Madrid 1.º (1015), PSG 2.º, City 3.º...	[OBSERVADO]
Blob comprimido	cabecera 0x1140364, zlib desde 0x11403D4	ver §H	[OBSERVADO]
Contratos (tabla I)	tras el blob (se desplaza con su tamaño)	ver §E	[PROBADO]
Negociaciones abiertas	tras el blob (0x1279F18 en la referencia), registros de 60 B	ver §E	[PROBADO]

Historial de traspasos	~0x12B430F, registros de 36 B	jugador, monto, fecha	[OBSERVADO]
Noticias / caja	0xBE34B4 ([reg][pid]...[tipo][fecha]) - 0xC8018C (36 B, [reg][pid]...[monto i32 x100])	el juego anota ahí cada salida y cada pago	[OBSERVADO]
Jugadores creados por el juego	en las plantillas de la IA: reg = 0xDB65xxxx, pid > 126.000	regens/canteranos; 771 al empezar, 1.127 un mes después; no están en el catálogo del parche	[OBSERVADO]

D. Alineación de los equipos de la IA (bloque de 600 B)

Campo	Dónde	Estado
Índice del bloque (= k)	+0 (u32)	[PROBADO]
ID option del equipo	+4 (u32)	[PROBADO]
Nombre del técnico	+8 (UTF-8)	[PROBADO]
Formaciones (coordenadas, varias copias)	+0x28...	[OBSERVADO]
Orden de formación: 40 índices de plantilla; 0-10 titulares, luego banca (7 o 12) y reservas; relleno 0xFF	+0x220	[PROBADO]
Roles (capitán, lanzadores): 6 índices de plantilla	+0x248	[PROBADO]

Reglas vistas en el juego: el juego lee las **primeras n entradas** (n = plantilla); si una es 0xFF sale un jugador en blanco «0». Formatos válidos: compacto (n + 0xFF), identidad (0...39) y «0...n-1, FF, n...». El que llega entra como **última reserva**; si se va un titular, una reserva ocupa su puesto. **Bloque 627 = el equipo del usuario**: lleva su ID (-11), de técnico el nombre del mánager, y su orden es el de verdad (§E). El 628 es su reserva (orden identidad) y el bloque del índice del club del usuario (154 en la referencia) es la copia «como equipo de la IA» (ID real 0xAD, orden identidad): no se usan.

E. El equipo del usuario: tablas, alineación y contratos

Qué	Cómo se localiza	Estado
12 tablas con un registro por jugador (A, A2, B, C, D, E, F, G, Hx3, M, I, J)	por ancla: los 5 primeros (reg, pid) de la plantilla a paso fijo, saltando huecos y fichas de gente que ya no está (hasta 8); pasos 24, 24, 24?, 44, 368, 192, 52, 108x3, 5628, 48, 16	[PROBADO]
Cómo las usa el juego	por (reg, pid), no por índice: admite huecos en medio; los fichajes nuevos van al final (en D, tras los 32 juveniles)	[PROBADO]
Registro de cada tabla	empieza 4 B antes de reg ([x][reg][pid]...) salvo contratos (28 B antes); vacío = reg 0/0xFFFF con pid 0	[PROBADO]
Qué hace el juego cuando un jugador se va	hueco en A, B, C, D, E, F, J, M (A: fecha vacía 0xFFFF en +16; B conserva +10; F +40 a 0; M también 0 en la palabra anterior al siguiente reg); compacta G, H y contratos; no toca A2 ni K al momento	[PROBADO] (réplica idéntica byte a byte)
A2	a 976 B de A; su campo anterior a reg es un float (valoración del último partido). Vacía (0xFFFF) en una carrera recién empezada: el programa no la exige	[OBSERVADO]

Ficha del último partido	16 B, solo los que jugaron: no es tabla de plantilla, no se toca	[PROBADO]
Orden de formación (40 B) + roles (6 B)	bloque 627 de alineaciones (+0x220 / +0x248): el primer bloque con el ID del usuario	[PROBADO]
Lista K (Estrategia)	16 B por puesto [flag, reg, pid, 0]; K0 flag 0, usados 0xC0-0xC6 (flags por jugador: p. ej. 0xC4), libre 0xC0/0xC1, después 0xC7. Se localiza por estructura; puede estar atrasada respecto al orden (el juego la rehace después)	[PROBADO]
ID interno de club	(ID option << 14) \ índice del bloque (City 0x2B409A = 173, 154); el bloque del usuario lleva -11 en vez del ID	[PROBADO]
Contratos (tabla I, 48 B; el registro empieza 28 B antes de reg)	[índice u8][ID club][tipo u8][inicio][-1][-1] \ [x u16][reg][pid][sueldo/100][cláusula/100][fin] - tipo 5 = contrato vigente - tipo 3 = oferta de otro club (sin inicio)	[PROBADO] (cifras = pantalla)
Contratos dobles	un jugador con ofertas tiene varios registros (5 + 3...); al salir se borran todos	[PROBADO]
Negociaciones abiertas (60 B, reg en +32)	[estado u16][ID club que oferta][banderas][0xFFFF][-1][-1][0xFFFF] \ [ID club del jugador][reg][pid][monto/100][monto/100][0xFFFF][0xFFFF][0]; vacío = -1/0xFFFF; al salir el jugador se borra y se compacta	[PROBADO]

F. Dinero del usuario (todo en u32 x 100 EUR)

Campo	Dónde	Estado
Presupuesto de fichajes actual	0xC7DD00	[PROBADO] (500 M visto en pantalla)
Presupuesto de fichajes inicial de temporada	0xC7DD10	[OBSERVADO]
Tope salarial	0xC7DD14	[PROBADO]
Presupuesto salarial en pantalla	= tope - Suma sueldos (tabla I, un contrato por jugador)	[PROBADO] (dos comprobaciones exactas)
Cómo lo localiza el programa	por ancla: el bloque está a 0x97D38 de la tabla A del usuario (estructura fija del motor, vista igual en 6 guardados, también en una carrera recién empezada) y se valida contra los contratos (tope >= Suma sueldos)	[PROBADO] (finanzas / fijarFinanzas en C++)
Clubes de la IA	no tienen presupuesto guardado; fichan por reglas	[NO]

G. Lo que NO está donde se esperaba

- Valor de mercado: no está en la ficha de 596 B; **sí** en la ficha de 156 B del blob (\$H).
- Habilidades de los jugadores en la LM: **no** están en el blob (se buscó con el mapa de bits de Player.bin y por fuerza bruta).

- Calendario de partidos: **no** aparece como rondas de pares de equipos ni como rejilla (se buscó exhaustivamente). Pista: los 10 IDs de partido por jornada del calendario del usuario. Experimento pendiente: guardar justo antes y justo después de un partido.

H. El blob comprimido (zlib)

Qué	Dato	Estado
Formato	desde 0x11403AC: 28 B de cabecera (0x0007458c...; +0x14 y +0x18 = sumas BE de los tramos) y 18 tramos zlib, cada uno con separador [descomprimido BE][comprimido BE][fin BE] (fin = rel + comprimido; 0 en el último)	[PROBADO]
Palabra de tamaño (0x11403A8)	tamaño de la zona menos 12 (el último tramo termina 12 B después)	[PROBADO]
Tamaño descomprimido	4.680.156 B, fijo	[PROBADO]
Escribirlo	se recomprimen solo los tramos cambiados (miniz nivel 9) y se rehacen separadores, sumas y tamaño; el archivo cambia de tamaño y lo de detrás se desplaza	[PROBADO] probado en el juego (prueba 17: valor de Sommer 77,7 M en pantalla)
Fichas de LM por jugador	en 0x1E + 156-reg (una por reg, 16.422)	[PROBADO]
- competencias inscritas	+10: 12 pares (id u16, banderas u16), libre = 0xFFFF; al fichar se añaden las del club	[PROBADO]
- 6 B que van a la tabla A al fichar	+0x4A	[PROBADO]
- sueldo anual /100	+0x56 (u32)	[PROBADO] (el juego crea el contrato de la tabla I desde aquí)
- valor de mercado /100	+0x5A (u32)	[PROBADO] (escrito y visto en pantalla)
- club anterior, fin e inicio del contrato	+0x5E, +0x66, +0x6A	[PROBADO]
- ¿experiencia/crecimiento?	+0x74 (u16), sube ~+11 por partido jugado	[HIPÓTESIS]
- ¿condición?	+0x7A (u8), 100 al inicio de temporada	[HIPÓTESIS]
Después de las fichas	registros de temporada por equipo con puntos de liga (+3 al ganar) y listas de inscritos por competición	[OBSERVADO]
Fecha actual de la partida	0xACC61C (u32 fecha), con [día del año][año][365] delante	[PROBADO]

H2. Fichar PARA el equipo del usuario (IA -> usuario) - [PROBADO] PROBADO EN EL JUEGO (pruebas 18 y 19)

Comparado con el fichaje de Sommer hecho por el propio juego y luego probado jugando un partido: - Plantillas: sale del club de la IA (lista compactada, su alineación con sustituto) y entra al final del equipo del usuario. - Dorsal: el pedido -> el de su club -> el de su selección -> el más alto libre (Sommer: el 1 ocupado -> 12, el de Suiza). - Tablas del usuario: registro nuevo en el primer hueco de cada una, copiando el de un compañero del mismo puesto; datos propios en A (6 B de la ficha + fecha de llegada), E (edad + historial de medias, que el juego inventa al fichar: 2 valores por año desde los 9), F (contador a 0), M (club, fechas, club de origen, bit 0x80000 «fichado esta temporada»). - Orden de formación +n y lista K con el flag del libre. Dinero: se descuenta el monto; el tope salarial nunca por debajo de los sueldos. - **Contrato: NO se escribe en la tabla I** - el juego lo crea solo desde la ficha del blob (con su cláusula). Si se escribe, queda

duplicado. - Blob: competiciones del club, sueldo, fechas y club anterior. - Resultado en el juego: Sommer jugó de titular, un solo contrato, presupuesto salarial positivo. - El juego **relee el archivo al cargar la ranura**: se puede cambiar un guardado con el juego abierto si esa ranura no está cargada.

I. Qué hace el juego por su cuenta (para no pelear con él)

- Tras un partido reescribe: cabecera 0x30, la zona posterior al blob (contratos, calendario, historial), fichas de 156 B de los que jugaron, puntos de liga; **no** corrige plantillas ni alineaciones editadas.
- Al avanzar el día hace fichajes de la IA (cientos de cambios de plantilla en el último día de mercado).
- Al salir un jugador del usuario: borra sus contratos y negociación, vacía o compacta sus registros en las tablas (ver §E) y pone a un suplente en su puesto; deja K y A2 para más tarde; anota noticia y pago en caja; recalcula las medias del equipo (bloque +0x5DA).
- Crea jugadores propios (regens, reg 0xDB65xxxx) y los reparte entre los clubes de la IA a lo largo de la temporada.

J. Player.bin (base de datos) - mapa ampliado (8 oct, 22:10)

Registro de 312 B; **NO** tiene el mismo orden que el jugador editado del option file (ese sí sigue el mapa de 4ccEditor, licencia zlib: pes20.cpp, fill_player_entry20). Posiciones en bits desde el inicio del registro (id u32 en +8):

Campo	Bits	Estado
19 habilidades de campo (6 bits, valor - 40)	ver §B	[PROBADO]
Potencia de tiro	358 (6, -40)	[PROBADO] (2026-10-09: Lamine 78 en la pantalla del juego = valor leído; las 15 cualidades visibles coinciden)
5 habilidades de portero	269, 300, 320, 326, 364 (6, -40)	[OBSERVADO] (40 en el 99,8 % de los jugadores de campo; altas en porteros). Falta saber cuál es cuál (actitud, atajar, despejar, reflejos, cobertura)
Posiciones jugables (2 bits: 0 no, 1 B, 2 A)	PT 350 - DFC 468 - LI 318 - LD 474 - MCD 414 - MC 456 - MI 466 - MD 460 - MO 464 - EI 472 - ED 476 - SD 478 (probable) - DC 470	[OBSERVADO] (la posición registrada vale 2 en el 99 %)
Pie malo uso / precisión, forma, resistencia a lesiones	454 (2) / 462 (2), 438 (3), 458 (2)	[HIPÓTESIS] (coinciden con el option file en los 21 editados)
Habilidades especiales + estilos COM	bits 480-531 (52 bits)	[HIPÓTESIS] en calibración (490, 496, 502, 522 solo en porteros: despejes/saques/penales de portero)
País	233 (9) -> Country.bin (1420 B por país: ID en los bits 10-18, nombre en español en +148)	[PROBADO]

Hallazgo importante: en ConmeGOL 26 la base que usa el juego **no es** CGP_database.cpk (19/8) sino la que sirve Sider desde SiderAddons\olmosjr23\Database\common\etc\pesdb\ (5/9): **5.894 jugadores distintos y 64 cambiados**. El catálogo del Mercado se generó con la base vieja -> hay que regenerarlo desde la de Sider.

Imágenes: minifaces en livecpk\Logos\common\render\symbols\player\<pid>.dds (2.638; DXT, 180x180) y escudos en ...symbols\flag\<6 dígitos del ID>_r_l.png (Barça: e_000108_r_l.png).

Guardado recién creado: una carrera guardada en cuanto se crea todavía no tiene fecha (1/1, día 0), ni alineación, ni contratos definitivos: el juego los rellena al empezar la temporada.

02 - Hacia el parche Phoenix (parche propio, vinculado a la web, a Phoenix Link y al Mercado)

Objetivo declarado (8 oct 2026): sacar un parche propio, el más ambicioso posible, que sea **uno solo con el ecosistema**: la web (liga, mercado, perfiles), Phoenix Link (salas) y Phoenix Sync (lo que entra al juego). Todo lo que se descubre auditando ConmeGOL y otros parches alimenta este plan. Esta página es el **mapa de lo que hay que dominar** y el estado real de cada pieza. Lo marcado [HIPÓTESIS] está por verificar: no se da por sabido.

A. De qué está hecho un parche de PES 2021 (anatomía)

Pieza	Dónde vive	Qué contiene	Cómo se fabrica / edita	Estado nuestro
Archivos CPK	<juego>\download*.cpk	paquetes CRI con todo el contenido: base de datos, caras, uniformes, estadios, balones, botines, logos, menús	se arman con herramientas CPK (CriPakTools / CPK File Builder); el juego los carga por DpFileList.bin	leemos la base (.cpk -> Player.bin) [PROBADO]; no armamos CPK todavía [PENDIENTE]
DpFileList.bin	<juego>\download\	lista de CPK que el juego carga y en qué orden (el último pisa a los anteriores)	«DpFileList Generator» o generador propio	[OBSERVADO] formato leído del ConmeGOL (ver parches/conmegol-26.md): 16 B + 48 B por CPK
Base de datos (common\etc\pesdb\)	dentro del CPK de base (ConmeGOL: CGP_database.cpk)	Player.bin (jugadores y habilidades, mapa de bits), Team.bin, PlayerAssignment.bin (plantillas), Competition*.bin, Coach.bin, Stadium.bin, Ball.bin, Boots.bin...	editores de base (PES editor / 4ccEditor) o escritura directa con el mapa de bits	Player.bin [PROBADO] (mapa de bits en 01 \$B); el resto [HIPÓTESIS]
Option file (EDIT00000000)	carpeta save de Steam	jugadores/equipos editados, plantillas, dorsales, tácticas, uniformes editados, competiciones editadas	el juego (modo Editar) o Phoenix Sync	plantillas, dorsales y tácticas [PROBADO]; uniformes/competiciones [HIPÓTESIS]
Caras y cuerpos	CPK (common\character0\model\character\face\real\<id>)	modelos .fmdl, texturas .ftex, face.fpk	Facemaker / Blender + herramientas de la comunidad	solo se reutilizan (viajan con el ID del jugador)
Uniformes	CPK (...uniform\team\<id>) + kit_config	texturas y configuración por equipo; también vía Sider (kserv)	Kit Studio / plantillas PSD + kit_config	[HIPÓTESIS]
Estadios, balones, botines, guantes, adboards, marcadores	CPK	modelos y texturas; muchos se sirven mejor por Sider (stadium server, ball server...)	herramientas de la comunidad	[HIPÓTESIS]
Logos, banderas, menús	CPK (Asset\..., common\menu\...)	emblemas de equipo y competición, fondos, menú	editores de imágenes + empaquetado CPK	[HIPÓTESIS] rutas exactas por confirmar
Sider (juce66)	carpeta del juego: sider.exe, sider.ini, modules*.lua	carga contenido suelto sin reempaquetar (LiveCPK: content\live-cpk\), módulos Lua (uniformes, estadios, marcadores, cámara, jugabilidad), opción save.folder (7.4.0)	se escribe en Lua con la API de Sider	se usará como vehículo [PROBADO] (decidido); módulos propios [PENDIENTE]

Guardado de Liga Máster (ML0000000N)	carpeta save	la carrera: 700 equipos, tablas del usuario, alineaciones, contratos, dinero, blob (calendario/resultados)	Phoenix Sync	ampliamente documentado en 01 §C-§I [PROBADO]
--------------------------------------	--------------	--	--------------	---

B. Lo que el ecosistema ya tiene y encaja

Pieza	Estado
La web guarda la verdad de la liga (clubes, plantillas, traspasos) y la firma (Ed25519)	[PROBADO] (docs/mercado-api.md)
Option file oficial publicado por el staff, con huella SHA-256 (/option/actual) y descarga verificada	[PROBADO] (PhoenixSync bajar)
Lista de cambios de liga (/liga/cambios), firmada, con versión incremental	[PROBADO] web - [PROBADO] Mercado (sincronizar, 8 oct)
Aplicar los cambios al option file (amistosos/salas) con alineación correcta	[PROBADO] (probado en archivo; prueba en juego pendiente)
Aplicar los cambios a la Liga Máster del usuario: usuario->IA e IA->IA	[PROBADO] (probado en juego: ranuras 9, 10, 13)
Aplicar a la Liga Máster: IA->usuario y agentes libres	[PENDIENTE] (quedan como «pendientes» en el informe hasta tener la referencia del juego)
Interruptores: option file / Liga Máster / ambos	[PROBADO] (Alcance)
Presupuesto del club del usuario desde fuera (p. ej. el que fija la web)	[PROBADO] (fijarFinanzas)
Catálogo del parche (jugadores, posiciones, habilidades) subido a la web	[PROBADO]
Equivalencias entre parches (jugar con otro parche sin romper la liga)	[PROBADO] (emparejamiento + huella)

C. Lo que falta dominar para un parche propio (por orden sugerido)

- **Empaquetar CPK y generar DpFileList.bin** desde Phoenix Sync (o un script). DpFileList.bin ya está entendido [OBSERVADO]; el CPK se puede **evitar casi del todo con LiveCPK de Sider** (así hace ConmeGOL: solo su base va en un .cpk de 3,7 MB; el resto, carpetas sueltas). [HIPÓTESIS] armar el .cpk de la base
- **Escribir la base de datos** (Team.bin, PlayerAssignment.bin, Competition*.bin) y no solo leer Player.bin: crear/renombrar equipos, cambiar plantillas de fábrica, competiciones y calendarios base. [HIPÓTESIS] (formatos por documentar; el mapa de bits de Player.bin ya está).
- **Uniformes y logos** con kit_config y emblemas: el mínimo visual para que un club de la liga «exista». [HIPÓTESIS]
- **Sider**: módulos propios (p. ej. cargar el option/plantillas de la sala en caliente, marcador con el nombre de la liga, save.folder por usuario para el «switcher»). [PENDIENTE]
- **Distribución y actualización**: el parche se baja/actualiza desde la web igual que el option oficial (huella SHA-256, descarga verificada, nunca se sobrescribe sin copia). [PENDIENTE] (la mecánica ya existe para el option).
- **Ligas propias**: fuera de la Liga Máster primero (competiciones del option file / base), dentro después (01 §C/§I). [HIPÓTESIS]

D. Qué auditar en cada parche que se estudie (para copiar lo bueno)

- Qué CPK trae y en qué orden los carga (DpFileList.bin).
- Cómo organiza la base (pesdb), cuántos jugadores/equipos/competiciones, qué IDs usa (rangos libres para crear).
- Si usa Sider y qué módulos (para no reinventarlos).
- Rarezas del option file (jugadores «Player Identidad», equipos vacíos, formatos de tácticas).

Regla: cada hallazgo se anota con su estado ([PROBADO] [OBSERVADO] [HIPÓTESIS] [NO]) y, si se probó en el juego, con su entrada en PRUEBAS.md.

03 - Lo que enseñan los demás parches (y cómo hacer el Phoenix el más completo y sin errores)

Investigación del 8 oct 2026: parches grandes de PES 2021, modo online, herramientas y formatos. Informes completos, con todas las fuentes, en investigacion/: - parches-pes2021.md: ficha de cada parche, las 14 causas de crash más reportadas con su solución y buenas prácticas. - modo-online.md: servidores oficiales, PESBUL, alternativas, Sider para resultados, Parsec desde Perú y arquitectura recomendada. - herramientas-y-formatos.md: cómo fabricar CPK, formatos de la base de datos, kits, caras, option file y memoria (Cheat Engine).

A. El mapa de los parches en 2026

Parche	Estilo	Lo que hay que copiar	Lo que hay que evitar
Football Life (SmokePatch; FL26 15/10/2025, FL27 a inicios de nov. 2026)	Juego «propio» sobre el motor: exe, lanzador, instalador y switcher propios	Lanzador que comprueba versiones; updates clasificadas según si rompen la carrera; ligas de 30 y Mundial de 48 parcheando el exe	Exe modificado sin online (PESBUL no funciona con FL)
Evoweb Patch (2024 v3.0)	Sider + base de datos	IDs reales de Konami para que cualquier facepack encaje	No convive con ninguna otra base de datos
ConmeGOL (27, sept. 2026)	Casi todo por Sider/livecpk; base en un .cpk de 3,7 MB	Base pequeña + contenido suelto (actualizar es copiar carpetas)	AIO de ~234 GB; sider.ini con módulos que faltan y pares que se pisan (ver ../sider/AUDITORIA-SIDER.md)
Sudamerican Project 2026	«Clásico» de CPK: 34 CPK, 13 de caras de 5-11 GB	Créditos y comunidad (Discord, 30+ colaboradores)	CPK gigantes; módulos cargados dos veces; netblock.lua (bloquea el online)
Gogosz (World 12.5 / CONMEBOL 4.5, oct. 2026)	AIO enorme (~200 GB)	Cobertura de ligas	El tamaño
PESBUL (beta, jul. 2026)	Servidor de la comunidad que imita al de Konami	Que el online «de verdad» es posible	Solo amistoso 1 vs 1; exige el juego de Steam sin modificar

Hueco libre: no existe un parche de PC dedicado a la **Liga 1 de Perú** y la zona andina; hoy solo aparece dentro de ConmeGOL. Ahí entra Phoenix.

B. Las causas de crash que más se repiten (y la regla Phoenix para cada una)

Causa	Regla Phoenix
Copias repack/modificadas, restos de otros parches, DpFileList para el DLC equivocado	Base única declarada: PES 2021 de Steam, exe 1.07.02 + Data Pack 7.0; el lanzador lo comprueba y no arranca si no coincide
Mezclar bases de datos (jugadores «sin nombre, media 40»)	Una sola base; Phoenix Sync escribe solo en la de Phoenix y detecta otras
Option file de otra versión	El EDIT lleva su versión; save.folder de Sider 7.4.1 para que Phoenix tenga su propio EDIT
Falta memoria virtual (pantalla negra con muchas caras)	El lanzador revisa el archivo de paginación y avisa; caras en DDS comprimido
Módulos Sider viejos, duplicados o que se pisan	Un solo sider.ini limpio, auditado; Sider fijado (7.4.1) y distribuido con el parche

Intel 12. ^a gen+ (DRM), drivers, antivirus, OneDrive	Lista de comprobación en el lanzador; hashes publicados; nada de .bat (lanzador .exe)
Cambiar el número de equipos de una liga (Evoweb: la Liga Máster se colgaba en la temporada 2)	No cambiar el número de equipos de una liga sin parchear el exe; el calendario está dentro del exe
Audio/mods de narración incompatibles	Addons con versión; se instalan de a uno y el lanzador comprueba la versión

Límites documentados de la base de datos (fuente de foro): **750 equipos, 40 jugadores por equipo, 30.000 jugadores.** (Cuadra con lo visto: 749 equipos en ConmeGOL y 29.997 jugadores en Player.bin.)

C. Modo online: lo realista

- Los servidores oficiales de Konami están **cerrados** (2022). PES 2021 de PC **no tiene modo LAN**: Radmin/ZeroTier/Hamachi solos no sirven.
- **PESBUL**: rehace el lobby de Konami (amistoso 1 vs 1). El partido va directo entre los dos PC. **Exige el juego de Steam sin modificar, sin Sider ni mods de jugabilidad**, y abrir los puertos UDP 5739-5740. Las dos carpetas de juego de la PC de Fralex no cumplen esas condiciones: llevan Sider con muchos módulos y archivos que no son de la instalación original de Steam (CPY.ini, steamclient64.dll). Es la explicación más probable de que PESBUL no conectara. Su base de código abierto es el servidor de **Nikow5** (licencia MIT).
- **Antecedente bonito**: en PES 6 hubo un servidor propio unido a una web de ligas, de código abierto (kinj1987/evo-league). Es la idea de «Phoenix online» hecha hace años.

Arquitectura recomendada para Phoenix online (se puede hacer ya): 1. La **web** crea el partido (liga, rivales). 2. **Phoenix Link** (Parsec) en el PC anfitrión deja entrar solo a esos jugadores y abre el juego con el option file oficial. 3. Un módulo **Sider** lee el marcador con match.stats() (marcador, penales, periodo y reloj; **no** da goleadores ni tarjetas) y lo deja en un archivo resultado.json (el «buzón», al revés que avisos.txt). 4. Phoenix Link sube el resultado a la web con una captura automática; los dos jugadores lo confirman. 5. El verificador revisa solo al anfitrión (sus archivos y su option file).

Así no hay desincronización (solo hay un juego corriendo) y no se depende de Konami ni de PESBUL. **Largo plazo (meses o años, investigación)**: servidor propio a partir del de Nikow5; haría falta entender el protocolo y el login de Steam.

Parsec desde Perú: el anfitrión necesita ≥ 10 Mbps de subida (30 recomendados); ping bueno < 30 ms, aceptable < 60 ms; ajustes para fútbol: 60 fps, 720p o 900p, H.265 si todos lo soportan, cable, V-Sync apagado, sin VPN. No funciona detrás de CGNAT (internet móvil).

D. Herramientas y formatos que ya tenemos documentados para fabricar el parche

- **Crear CPK**: the4chancup/pes-file-tools (cpk.py, unas 250 líneas): archivos desde 0x800, cada uno alineado a 0x800, sin comprimir, ordenados por nombre en mayúsculas; tablas en big-endian cifradas con XOR (empieza en 0x5F, x 0x15 por byte). Fácil de pasar a C++ (sin licencia declarada: usar como referencia y escribir código propio).
- **DpFileList.bin**: filas de 48 B con el nombre del CPK en +0x10 (ya entendido en parches/conmegol-26.md).
- **WESYS** (envoltorio de los .bin de la base): cabecera de 16 B (3 bytes de versión, «WESYS», tamaño comprimido, tamaño original) + zlib.
- **Tamaños de registro** (pes-db-generator, GPL-3): Team 1532, Coach 100, PlayerAssignment 16, CompetitionEntry 12, PlayerAppearance 60. **Sin documentación pública**: Competition, CompetitionKind, CompetitionRegulation, Stadium, Derby, Tactics, Ball, Boots, Glove -> hay que descifrarlos comparando antes/después (como hicimos con la Liga Máster).
- **Kits**: TeamColor.bin 16 B por equipo; UniColor.bin 85 B por equipo; UniformParameter documentado; kit config de 0x78 B (de kitinfo.cpp de Sider).
- **Caras**: pes-file-tools (.fpk/.ftex/.fsop), pes-fmdl-blender 0.7.2 (mayo 2026), GzsTool/FtexTool de Atvaark (MIT).
- **Option file**: el mapa de 4ccEditor (C++, licencia zlib) coincide **exactamente** con el nuestro (jugadores 0x7C x 0x138, equipos 0x8ED2FC x 0x24C, plantillas 0x9D4648 x 0x11C, tácticas 0xA09880 x 0x274): confirma nuestro trabajo.
- **Liga Máster**: no existe ningún editor de guardados de código abierto ni el formato documentado en público: **lo nuestro (01-MOTOR-PES2021.md) es, hasta donde se encontró, lo más completo que hay.**
- **Memoria (para el puente en vivo)**: tabla de Cheat Engine de xAranaktu (MIT): presupuesto de fichajes en [rdi+0x016ECBF4], salarial en +0x016ECC08; jugador en RAM de ~0x17C B (sueldo +0x15C, valor +0x174,

contrato +0x138).

- **Sider 7.4.1:** código en GitHub (pes-modding/sider7) - leer, no copiar (licencia). Novedad clave: save.folder.

E. Receta del Phoenix Evolution Patch (propuesta)

- **Base fija:** PES 2021 de Steam, exe 1.07.02 + DP 7.0, Sider 7.4.1 incluido, save.folder propio.
- **Núcleo pequeño:** base de datos Phoenix en un .cpk chico (como ConmeGOL) + option file oficial publicado por la web (ya existe).
- **Contenido por Sider/livecpgk**, en addons con versión (caras, kits, estadios, narración), con **un sider.ini auditado** sin duplicados.
- **Lanzador propio** (Phoenix Link): comprueba exe/DLC/Sider/addons, archivo de paginación, antivirus y OneDrive; muestra avisos de update.
- **Mercado y Liga Máster** siempre sobre la base Phoenix, con copia de seguridad automática antes de cada escritura (ya lo hacemos).
- **Online:** modelo Parsec + web + buzón Sider (sección C).
- **Identidad:** Liga 1 y ligas andinas como punto fuerte; IDs reales de Konami para que los facepacks encajen.
- **Pruebas antes de cada versión:** instalación limpia; arranque, amistoso, Liga Máster (temporada completa y mercado), copa, Ser una Leyenda, edit, option file y partido por Phoenix Link. Todo anotado en PRUEBAS.md.

04 - Informe estratégico: ConmeGOL y Sudamerican por dentro, y cómo superarlos

Fecha: 8 oct 2026. Basado en el análisis de las dos instalaciones de la PC de Fralex (bases de datos extraídas y comparadas, los dos Sider auditados, estructura de carpetas, logs) y en la investigación de 03-INVESTIGACION-PARCHES.md. Leyenda de viabilidad: [VERDE] **ya se puede** (con lo que dominamos) - [AMARILLO] **se puede con investigación** (camino conocido) - [ROJO] **frontera** (nadie lo ha hecho; puede no salir).

1. Los dos parches por dentro

1.1 Base de datos (lo que define «el mundo»)

	ConmeGOL 26	Sudamerican Project 2026
Archivo base	CGP_database.cpk (3,7 MB)	SP_Subsc.cpk (3,9 MB) + SP_Subsc_WC.cpk (Mundial, no cargado)
Competiciones	90	80
Equipos	749 (techo del juego: 750)	726
Jugadores	29.997 (techo: 30.000)	29.323
Asignaciones a plantillas	20.398	17.214
Archivos «4» (variantes de Konami)	no	sí (Team4, Player4, Competition4...)
Idioma del option file	español	portugués
Equipos con el mismo ID en los dos	467 (274 con el mismo nombre)	

Lo que trae cada uno y el otro no: - **Solo ConmeGOL:** Libertadores, Sudamericana y Recopa con nombre CONMEBOL; ligas de **Venezuela, Uruguay (copa y 2.ª), Paraguay (copa), Costa Rica, Honduras, MLS y Arabia**; 2.ª de Colombia y Ecuador; «Clásicos» y «Leyendas». - **Solo Sudamerican:** **Alemania y Países Bajos** completas, Europa League y Supercopa de Europa, 2.ª de Inglaterra, Supercopa de España, copas de Ecuador/Paraguay/Uruguay/México/EE. UU., y **2.ª de Perú con 14 equipos** (en ConmeGOL está vacía).

Perú (lo que más nos importa): - **Liga 1 2026 (18 equipos):** los dos la tienen igual: Alianza Atlético, Alianza Lima, Atlético Grau, FC Cajamarca, Cienciano, Comerciantes Unidos, Cusco FC, Garcilaso, Moquegua, Juan Pablo II, Los Chankas, Melgar, Sport Boys, Sport Huancayo, Sporting Cristal, ADT, UTC y Universitario. - **Copa (32):** los dos la tienen (con 2.ª división incluida). - **Liga 2:** solo Sudamerican (Cantolao, ADA Jaén, Alianza Universidad, Ayacucho, Bentín Tacna, Mannucci, Comerciantes FC, Estudiantes CNI, San Marcos, Piratas, Santos Nazca, Unión Minas, César Vallejo, San Martín). - **Supercopa:** ConmeGOL Universitario-Alianza; Sudamerican Universitario-Cusco.

Lección: los dos van **pegados al techo** del juego (750 equipos, 30.000 jugadores). Cada liga nueva obliga a **sacar** otra. Phoenix tiene que **elegir** qué ligas trae (calidad antes que cantidad) y dejar espacio libre para crecer.

1.2 Cómo entregan el contenido

	ConmeGOL	Sudamerican
Modelo	Casi todo suelto por Sider (livecpk + content)	Clásico de CPK: 34 CPK en DpFileList
Caras	por Sider	13 CPK de 5-11 GB cada uno
Estadios	stadium-server: ~9 GB en más de 11.000 archivos	stadium-server: 313 carpetas
Kits	kit-server por país	kit-server: 36 países, 157 equipos
Balones	ball-server (34)	ball-server (1.582)
Otros pesados	Turfs 2,4 GB, narración de estadio 1,8 GB, gráficos «next gen» 1,7 GB, cuerpos PRDX ~1 GB (6.000+ archivos)	narradores en CPK (Bambino Pons, Solabarrieta, Miguel Simón) y en Sider (Closs, Palma, Barril, Tapia, Romano, Vignolo)

Archivos diminutos	slotkits: ~9.900 archivos casi vacíos	-
Sider	7.3.3, 49 módulos activos	7.3.4, 56 módulos (normal) / 66 líneas (Mundial)
Lanzador	switcher que copia carpetas encima	7 .bat (uno por letra de disco)

(El recuento de archivos de ConmeGOL es parcial: la carpeta es tan grande que leerla entera por el puente tarda horas; las cifras de arriba son las ya contadas.)

1.3 Sus estrategias para ser estables (lo que hacen BIEN)

- **Base de datos pequeña y aparte** (3,7-3,9 MB): la parte delicada cabe en un archivo; el resto es decorado.
- **Sider como capa de contenido** (sobre todo ConmeGOL): actualizar = copiar carpetas, sin reempaquetar CPK gigantes.
- **Variantes por switcher** (ConmeGOL: normal / Clásicos / Perú-Chile-Uruguay B; Sudamerican: normal / Mundial): cada modo tiene su propia base y su propio sider.ini, en vez de meter todo junto.
- **Módulos de la comunidad muy probados** (kserv, StadiumServer, BallServer...): no reinventan lo básico.
- **Comprobaciones de identidad**: los módulos CGP solo se activan si encuentran ciertos equipos en ciertas competiciones (un «candado» para no funcionar con otra base).
- **Comunidad y créditos** (Sudamerican: más de 30 colaboradores, Discord): mantener un parche es trabajo de equipo.

1.4 Sus fallos (lo que hacen MAL, comprobado en tu PC)

Fallo	ConmeGOL	Sudamerican	Consecuencia
camera.lua + DynamicWideCam.lua parchean la misma instrucción	sí (log)	sí (log, mismas direcciones)	Escriben encima de datos del juego; cierres raros
Escrituras a ciegas (GFX_lod)	sí	sí	Ídem
Módulos que no existen en sider.ini	11	3	Ruido; señal de desorden
Módulos que fallan al arrancar	-	3 (randomMenu, TurfLoader, LogoHD)	Funciones que el usuario cree tener y no tiene
Módulos cargados dos veces	-	MenuServer; en el Mundial env, etrace, camera, netblock	Doble parcheo
Módulos de prueba activos (env, etrace, jittest, zlibtest)	env	varios	Logs gigantes, más carga
Módulos que no hacen nada / de otra versión (Chatty para FL26, scoreboard EPL, Anti-Cheat placebo, goalscreams)	sí	parcial	Falsa sensación de función
netblock.lua	-	sí	Bloquea cualquier online
Contenido duplicado entre modos (cada variante copia todo)	sí (~234 GB el pack completo)	sí (dos Sider casi iguales)	Pesado, lento de actualizar
CPK de caras de 5-11 GB	-	sí	Actualizar una cara = bajar 10 GB
Miles de archivos diminutos (slotkits, cuerpos PRDX)	sí	-	Cada archivo cuesta una búsqueda de Sider
Instalación por .bat con letra de disco	-	sí	Frágil; el antivirus desconfía
Lanzador que pisa carpetas sin avisar	sí	-	Perdemos cambios (nos pasó con phoenix.lua)
Copias del juego no originales	sí	sí	Sin PESBUL; riesgo de cargas infinitas

2. Cómo superarlos en estabilidad ([VERDE] todo esto ya se puede)

- «**Sider limpio**» **certificado**: cada módulo que entre al parche pasa la auditoría (memoria, archivos, teclas, choques). Prohibido: dos módulos que toquen lo mismo, escrituras sin comprobar, módulos de prueba, duplicados.
- «**Phoenix Doctor**»: la auditoría que hicimos a mano, convertida en **herramienta automática** dentro del lanzador: lee sider.ini y sider.log, detecta módulos que faltan, duplicados, fallos de arranque y choques conocidos, y propone la corrección. (Ningún parche tiene esto.)
- **Lanzador único** (Phoenix Link): sin .bat; comprueba versión del juego, Sider, addons, archivo de paginación y espacio; **nunca pisa sin copia**; muestra qué cambió.
- **Una base, un sider.ini por modo**, generados por el programa (no a mano) y con su huella SHA-256 comprobada en cada arranque.
- **Pruebas automáticas** antes de publicar (ya tenemos 271 para el Mercado) + la lista de pruebas en el juego de PRUEBAS.md.
- **Reglas de base de datos**: no cambiar el tamaño de las ligas sin parche del exe; dejar huecos libres de IDs; usar IDs reales de Konami (los facepacks encajan solos).

3. Cómo superarlos en rendimiento ([VERDE]/[AMARILLO])

- **Menos archivos, más grandes** ([VERDE]): juntar los miles de archivos diminutos (slotkits, cuerpos) en pocos paquetes; cada archivo suelto es una búsqueda de Sider por cada carga.
- **Contenido solo de lo que juegas** ([VERDE]): addons por liga (si no juegas la MLS, no cargas sus caras ni estadios).
- **Texturas bien comprimidas** ([VERDE]): DDS con compresión adecuada y tamaños razonables (no 4K donde no se ve) -> menos VRAM, menos archivo de paginación, menos pantallas negras.
- **Módulos ligeros** ([VERDE]): nada que trabaje en cada fotograma sin necesidad; nada que mire el disco por cada archivo (como hace real-coach-team); un solo módulo Phoenix en vez de diez pequeños.
- **Caché de búsquedas de Sider** bien ajustada ([AMARILLO]): lookup-cache, cache.size y el orden de las raíces cpk.root (las pequeñas y específicas primero).
- **Medición** ([AMARILLO]): un módulo que mida tiempos de carga y FPS y los anote, para comparar versiones con números y no a ojo.

4. Funciones nuevas: hasta dónde podemos llegar

[VERDE] Ya se puede (con lo que ya probamos)

Función	Qué es	Por qué nadie la tiene
Mercado conectado a la web	Fichajes reales de la liga (web firmada) aplicados al option file y a la Liga Máster	Ya funciona (pruebas 14-19); los parches solo traen plantillas fijas
Liga Máster «viva»	La web fija presupuestos, mete fichajes o ventas en tu carrera entre partidos	Nadie había descifrado la Liga Máster (no hay editor público)
Option file por sala	Phoenix Link carga el option file de la liga/sala elegida automáticamente	Los parches usan un único EDIT para todo
Avisos en el juego	Mensajes de la web dentro del juego (puente avisos.txt)	Prueba 1 hecha, falta probarla en el juego
Resultado automático	Módulo que lee el marcador (match.stats()) y lo deja en un archivo para que Phoenix Link lo suba	Las ligas actuales usan capturas a mano
Phoenix Doctor	Diagnóstico automático del Sider y del juego	Ver §2
Verificador del anfitrión	Antes de una partida de liga, se comprueban las huellas del parche y del option file	Base: el verificador que ya planeabas
Partidos «de liga» en amistoso	Hacer que un amistoso se vea como partido oficial (menú, marcador, intro de la liga Phoenix), como hace CustomExhibitionMatch.lua	Existe para Euro/Asia; nadie lo usa para una liga propia

[AMARILLO] Se puede con investigación (el camino es conocido)

Función	Qué hace falta
Estadísticas completas a la web (goleadores, tarjetas, posesión)	match.stats() no las da: leerlas de la memoria (tabla de Cheat Engine de xAranaktu como punto de partida)
Cambios en vivo dentro de la Liga Máster (sin salir del juego)	Mapa de la Liga Máster en la memoria (presupuestos ya localizados por xAranaktu)
Fabricar nuestros propios CPK	Portar el formato de pes-file-tools a C++
Competiciones propias (Liga Phoenix, copas de la web)	Descifrar Competition*/CompetitionRegulation comparando antes/después (como hicimos con la Liga Máster)
Calendario de la Liga Máster desde la web	Encontrar dónde se guarda el calendario (pista: los 10 IDs de partido por jornada)
Actualizaciones por partes (solo lo que cambió)	Paquetes pequeños + manifiesto con huellas (la web ya publica el option file así)
Ligas de más de 20 equipos o formatos nuevos	Parches del exe localizados por patrón (como hace FL)
Informe de errores	Que el lanzador, con permiso, mande sider.log cuando el juego se cierre, para arreglar fallos de todos

[ROJO] Frontera (nadie lo ha hecho; puede no salir)

Idea	Por qué es difícil
Liga Máster online asíncrona: varios usuarios, cada uno con su club en su propia carrera, compartiendo un mismo mercado y una misma tabla en la web	Hay que sincronizar las carreras (resultados de la IA, calendario, fichajes) sin un servidor dentro del juego. Con lo que ya sabemos de la Liga Máster es la idea más original y la más cercana.
Estado del mundo real en el juego: tabla, lesiones o forma de la Liga 1 real aplicadas cada semana	Fuentes de datos, y escribir forma o estado en la base o en la memoria
Servidor propio estilo PESBUL	Protocolo y login de Steam; meses o años de trabajo; PESBUL aún no tiene 11 contra 11
Espectadores en vivo: marcador y datos del partido en la web mientras se juega	Leer memoria en vivo + subir datos sin afectar al juego
Árbitro/VAR o reglas propias de la liga	Lógica del juego dentro del exe: requiere parches de código

5. Hasta dónde se puede «jugar» con el motor (límites honestos)

- **Archivos de la carpeta save** (option file, Liga Máster): **control casi total** - ya lo demostramos.
- **Base de datos (pesdb)**: control total en cuanto descifremos los archivos que faltan; límites de 750 equipos y 30.000 jugadores.
- **Contenido (caras, kits, estadios, menús, sonidos)**: control total vía Sider/CPK.
- **Memoria del juego en vivo (Sider + Lua)**: se puede leer y escribir; el riesgo crece con cada escritura -> solo con patrón + verificación + pruebas.
- **Reglas «cableadas» en el exe** (calendarios, tamaños de liga, IA, físicas): solo con parches de código, que se rompen si cambia el exe.
- **Red dentro del juego**: Sider no trae red; **todo lo online pasa por Phoenix Link** (fuera del juego). Es una limitación y a la vez una ventaja: el juego no se expone a Internet.

6. Ruta completa propuesta

Fase	Qué	Entregable
------	-----	------------

0. Terreno	Instalación limpia de referencia (Steam, 1.07.02 + DP 7.0, Sider fijado) + partido de referencia del Sider actual	Base de pruebas confiable
1. Núcleo	Base de datos Phoenix (partir de lo mejor de cada parche: Liga 1 + Liga 2 de Sudamerican + Copa, con permiso de sus autores) - option file oficial de la web - sider.ini limpio	Versión 0.1 (jugable, pequeña, estable)
2. Herramientas	Fabricador de CPK y DpFileList en Phoenix Sync - Phoenix Doctor - lanzador	Instalar/actualizar sin .bat
3. Conexión	Avisos (prueba 1) -> resultado automático -> verificador del anfitrión -> option file por sala	Primeras partidas de liga «online» con amigos
4. Contenido	Addons por liga (caras, kits, estadios) con versión; permisos y créditos	Parche completo, modular
5. Más allá	Competiciones propias, estadísticas completas, Liga Máster viva, y la gran apuesta: Liga Máster online asíncrona	Lo que nadie tiene

Créditos y permisos: casi todo el contenido visual de los dos parches es obra de terceros (facemakers, kitmakers, Olmos Jr 23, Afandix...). Para publicarlo dentro de Phoenix hace falta su permiso y su crédito. Lo propio de Phoenix (base de datos, herramientas, conexión con la web) es 100 % de Fralex.

7. Proyección a futuro: cuando los parches se actualicen (y lleguen otros)

Se esperan pronto: **ConmeGOL** (la versión instalada es la **26**, base del 19/8/2026; ya existe la **27**, sept. 2026) y **Sudamerican** (en una semana). Más adelante, **FL27** (inicios de noviembre) y otros. La idea: que cada actualización **no rompa nada nuestro** y que además **nos enseñe algo**.

7.1 Lo que suele cambiar en una actualización de parche

Qué cambia	Cómo afecta a Phoenix
Base de datos (fichajes, equipos nuevos, IDs que se mueven, competiciones nuevas)	El catálogo del Mercado queda viejo -> regenerarlo y subirlo a la web; revisar si cambian IDs de equipos/jugadores
Option file nuevo	Las plantillas cambian; el option file oficial de la web tiene que rehacerse sobre la nueva base
Módulos Sider nuevos o actualizados, sider.ini nuevo	Volver a auditar (choques, escrituras a ciegas); el switcher puede pisar lo que hayamos instalado
Carreras de Liga Máster	Muchos parches piden empezar una carrera nueva tras actualizar la base; las carreras viejas pueden dar plantillas raras
Estructura (más CPK, nuevas carpetas livecpk)	Revisar DpFileList (orden, cantidad) y espacio en disco

Lo que NO debería romperse: Phoenix Sync localiza todo **por anclas** (busca la forma de los datos, no posiciones fijas) y valida antes de escribir; si algo cambió de forma, **se frena** en vez de estropear el archivo. Esa decisión de diseño es justo para este momento.

7.2 Huella del «antes» (hecha hoy, 8 oct 2026, 21:15)

Guardada en la PC en _PhoenixMercado_prueba_versiones\2026-10-08\ (y la lista en datos/huella-2026-10-08.txt): - ConmeGOL: CGP_database.cpk (md5 d2d62b4d..., 3.699.240 B, 19/8/2026), DpFileList.bin, sider.ini (md5 80ad499f...), option file (md5 027b0fa2...), lista de módulos y CPK con tamaño y fecha. - Sudamerican: SP_Subsc.cpk (11/5/2026), SP_Subsc_WC.cpk, DpFileList.bin, los dos sider.ini, option file (md5 6c612a4c...). - El mismo PES2021.exe en los dos (md5 d4760962...). Con esto, después de actualizar se puede decir **exactamente** qué cambió.

7.3 Protocolo del «día de actualización» (para mañana y para siempre)

Antes (Fralex): 1. Copiar la carpeta save del parche (option file y carreras) a un lugar seguro. 2. Avisar: «voy a actualizar ConmeGOL / Sudamerican».

Después (Claude, con la PC conectada): 1. Nueva huella y **comparación** con la anterior: qué archivos cambiaron. 2. **Base de datos**: equipos que entran/salen, IDs que cambian, competiciones nuevas, jugadores movidos (con atención a Perú). 3. **Sider**: módulos nuevos o cambiados -> auditoría solo de esos; comprobar si siguen los choques de cámaras, etc. 4. **Option file**: comparar plantillas. 5. **Mercado**: regenerar el catálogo, correr las pruebas (PROBAR2.bat), subir el catálogo nuevo a la web. 6. **Liga Máster**: probar que el programa lee una carrera nueva de la versión nueva (y decidir qué pasa con las viejas). 7. Anotar todo en parches/<parche>.md como «**changelog técnico**» (lo que el autor no cuenta), en PRUEBAS.md y en Drive.

7.4 Automatizarlo: el «Auditor de parches» (fase 2)

Lo que hoy hacemos a mano se convierte en un comando de Phoenix Sync: - auditar <carpeta del juego> -> huella completa (base, DpFileList, Sider, option file), comparación con la última, informe en Markdown. - Comparador de bases de datos (equipos, competiciones, plantillas) y **tabla de equivalencias de IDs entre versiones** (reutilizando el emparejamiento ConmeGOL <-> Sudamerican que ya existe). - Auditoría automática de sider.ini y módulos (Phoenix Doctor). - Un **registro de parches** (nombre, versión, carpeta del juego, carpeta save, número de Steam) para que el Mercado y Phoenix Link sepan siempre con qué parche y qué versión trabajan, y no mezclen archivos (ConmeGOL y Sudamerican comparten la carpeta de documentos).

7.5 Parches futuros (FL27, Gogosz, nuevas versiones)

Cada parche nuevo que entre a la PC recibe la misma ficha: identificación, base de datos (comparada con las demás), contenido, Sider auditado, estrategias, fallos, y qué copiar o evitar. Con varias fichas se ve **la tendencia** (qué hace la comunidad cada temporada) y Phoenix puede adelantarse.

7.6 El propio Phoenix, preparado para su futuro

- **Versiones con número claro** y cada actualización marcada: «no afecta a tu carrera» / «necesita carrera nueva» / «se aplica la próxima temporada» (como FL).
- **Huella publicada** de cada archivo (como el option file oficial de la web) y lanzador que comprueba y repara.
- **Las carreras y los datos de la web sobreviven** a las actualizaciones: la web guarda la verdad (fichajes, presupuestos) y el Mercado puede **reaplicarla** sobre una base nueva.

05 - La «puerta en vivo»: cambiar la base del juego sin reiniciar

Descubierto el 2026-10-09 (mañana, Lima). Probado en el juego de Fralex (ConmeGOL Patch 26, Sider 7.3.3).
Diario completo de cada prueba: ../PRUEBAS.md (Pruebas 1-4 de Sider). Este documento resume **qué se aprendió, qué abre y qué queda para el futuro**.

1. El método ([PROBADO] PROBADO)

Nuestro libro. Phoenix Sync genera un archivo de la base (por ejemplo, Player.bin):

- parte de la base que sirve el parche (hoy olmosjr23\Database), descomprimida;
- le aplica los cambios;
- lo vuelve a comprimir como Konami: cabecera WESYS de 16 bytes (ff 10 81 57 'WESYS', tamaño comprimido u32, tamaño real u32) + zlib.
- La compresión no tiene que ser idéntica a la original: el juego acepta zlib nivel 9.

Sider lo entrega primero. Se pone en la raíz SiderAddons\livecpk\Phoenix-DB\common\etc\pesdb\ y en sider.ini va la línea cpk.root = ".\livecpk\Phoenix-DB" **antes** de cpk.root = ".\olmosjr23\Database". Gana el primero que tenga el archivo.

- Va en la raíz del juego y en la carpeta del modo (ConmeGol Extras\<modo>\SiderAddons), porque el switcher copia esa carpeta encima con robocopy.
- **Con el juego abierto** se reemplaza el archivo de forma atómica (.tmp y luego renombrar).
- **En el juego: Editar -> Cargar** (cargar EDIT00000000, **sin guardar**).
- **En el siguiente partido o pantalla nueva** se ven los cambios.

Prueba: Lamine Yamal (ID 162114), Velocidad 99 -> 95 -> 90. Cada lectura del juego se identificó con una «huella» (suma cada 64 bytes del trozo que entrega Sider por livecpk_read).

2. Lo que NO funciona (para no repetirlo)

Intento	Resultado	Motivo
Leer memoria directo (memory.read, búsqueda por regiones a lo largo de varios cuadros)	[NO] crash 0xC0000005 en sider.dll	Otros hilos (ReShade, DLSS) liberan memoria en cualquier momento. Regla: copiar con ReadProcessMemory, que falla sin cerrar el juego.
Buscar la ficha exacta de 58 bytes	[NO] 0 coincidencias	El juego cambia bits al cargar (forma, lesión). Hay que buscar por ID y verificar cualidades.
Escribir la Velocidad en las copias de memoria (WriteProcessMemory)	[NO] la pantalla no cambia	Las copias de 312 bytes se rehacen desde el archivo; la pantalla lee otra representación.
«Datos Actual. en vivo» (actualización nativa de Konami)	[NO]	«Los servicios en línea finalizaron el 25/08/2022»: depende del servidor de Konami.
Solo entrar y salir de Editar	[AMARILLO] lee la base y la descarta	Editar trabaja con un borrador propio. Hace falta Cargar.
Shift+R (recargar módulos de Sider)	[AMARILLO]	Solo recarga módulos que ya estaban activos. Uno que falló al iniciar necesita reiniciar el juego.

Otras lecciones del Lua de Sider 7.3.3: - **No hay pcall.** Si init falla, el módulo no se activa. Para contar errores se usa una «bandera». - **ffi.cdef no deja redeclarar.** Tras Shift+R el Lua es el mismo, así que se usan nombres con versión (phx06_* + __asm__("NombreReal")). - **ctx.sider_dir** termina en \.

3. Cuándo lee el juego cada archivo de pesdb (espía livecpk_read)

- **Al arrancar:** todos, dos veces (arranque y «Datos del sistema: Cargando»).

Al entrar a Editar y al pulsar Cargar:

- **sí relee:** Player, PlayerAssignment, Team, Coach, Competition, CompetitionEntry, CompetitionRegulation, Country, Stadium, Boots, Glove;
- **no relee siempre:** Tactics, PlayerWeekly y otros. Revisar en cada prueba.
- **Al entrar a la Liga Máster:** lee InstallVersionPlayer.bin, pero no Player.bin.
- **No relee:** al abrir la pantalla de habilidades, al armar un amistoso ni al cambiar «Datos Actual. en vivo».

4. El universo que abre

Archivo que se relee	Qué permitiría cambiar en vivo	Estado
Player.bin	Stats, habilidades, posiciones, altura, edad, nacionalidad	[PROBADO] Probado (Velocidad)
PlayerAssignment.bin	Fichajes y traspasos entre clubes (el jugador cambia de plantilla)	[VERDE] Mismo método, por probar
Coach.bin	Cambios de entrenador	[VERDE] Por probar
Team.bin	Datos de los clubes	[VERDE] Por probar
CompetitionEntry.bin	Ascensos, descensos, qué equipos juegan cada liga	[VERDE] Por probar (ojo: los módulos CGP comprueban equipos en ciertas competiciones)
Stadium.bin, Boots.bin, Glove.bin	Estadios, botines y guantes	[VERDE] Por probar
PlayerWeekly.bin / TeamWeekly.bin	Forma semanal propia (las flechas), al estilo de la actualización en vivo de Konami	[AMARILLO] Por investigar (la opción nativa pide el servidor)

Productos que salen de ahí: 1. **Mercado de fichajes real:** fichajes confirmados en la vida real o hechos en la web, aplicados con Editar -> Cargar. 2. **Phoenix Weekly:** stats ajustadas cada semana según el rendimiento real, con reglas fijas y aprobación de Fralex. Sin copiar notas de otros juegos. 3. **Liga online justa:** todos con el mismo archivo, verificado por huella SHA-256. 4. **Eventos:** jugador de la semana, castigos por lesión o suspensión, homenajes. 5. **Liga Máster conectada:** las habilidades de la LM salen de Player.bin (no están en el guardado). Por confirmar en la carrera.

5. Para el futuro (deseos de Fralex, aún no son prioridad)

Crear jugadores que no existen (juveniles, jugadores reales que faltan, jugadores inventados para la liga).

- **Hipótesis:** agregar registros nuevos (312 bytes, con un ID libre) a Player.bin y su fila en PlayerAssignment.bin.
- **Falta investigar:** rango de IDs seguros, límites de tamaño, PlayerDeleteList.bin, InstallVersionPlayer.bin y cómo reacciona la Liga Máster.
- **Caras:** carpeta face\real\<ID>\ y minicara symbol\player\<ID>.dds. Fralex quiere aprender a hacer faces; Claude le dará el ID exacto y la carpeta de cada jugador.
- **Avisos con aspecto nativo:** imágenes de los menús generadas por Phoenix Link y servidas por Sider (por ejemplo, una tarjeta «FICHAJE: ...» con la cara del jugador), en lugar del overlay de Sider.

6. Cuidados obligatorios

- **Cuando el parche se actualiza** (ConmeGOL o SP), la raíz Phoenix-DB **tapa** su base: hay que **regenerar** nuestros archivos desde la base nueva (Auditor de parches) **antes** de jugar. Si no, se servirían datos viejos.
- **No pulsar «Guardar» en Editar** durante las pruebas.
- **Nunca escribir memoria** durante un partido. El método por archivo no la necesita.
- **Respaldos en el PC de Fralex:** sider.ini.respaldo-phoenix-20261009 (módulo phoenix.lua) y sider.ini.respaldo-phoenixdb-20261009 (raíz Phoenix-DB). Variantes de prueba en _PhoenixMercado_prueba\db\.
- **Estado al cerrar la prueba:** Phoenix-DB contiene v90, que son los datos originales (solo recomprimidos).

Parche: ConmeGOL Patch 26 (PES 2021) - auditoría del 8 oct 2026

Identificación

Qué	Dato
Nombre interno usado por Mercado	conmegol-26
Carpeta del parche	D:\Frank\Games_\Conmegol Patch\
Base de datos (CPK)	ConmeGol Extras\ConmeGOL Patch 26\download\CGP_database.cpk (dentro: Player.bin)
Carpeta de guardados	...\KONAMI\leFootball PES 2021 SEASON UPDATE\239200\save (número de Steam 239200)
Comparte la carpeta de guardados con	Sudamerican Project (misma carpeta save: cuidado con mezclar)
Texto que muestra el juego en Cargar	Manchester City FC / Premier League - fecha - competición
Nombre del parche en la LM	«Conmegol» (no «Conmebol»)

Tamaños (sirven para detectar una versión nueva)

Qué	Valor
Catálogo (option file + CPK)	711 clubes, 18.304 jugadores (7 lotes para la web)
Option file: equipos con plantilla	749 (u16 en +100)
Option file: jugadores editados	21 (u16 en +96)
Liga Máster: bloques de equipo	700 (627 con nombre; selecciones desde el índice 528)
Liga Máster: bloques de alineación de la IA	629
Liga Máster: fichas de 596 B	5.759
Blob: fichas de 156 B	16.422
Guardado descifrado (datos)	19.759.355 B al inicio (4/8/2026) -> 19.795.679 B el 31/8/2026 (crece con el blob)

IDs y referencias útiles (índice de bloque en la LM - ID en el option file)

Equipo	Índice	ID option	Nota
Manchester City (equipo del usuario)	154	173 original; en la LM figura como -11 (0xFFFFFFFF5)	su alineación va en el bloque 627
Real Madrid	135	109	
Liverpool	153	-	
Santos	66	1254	
Boca Juniors	-	139	prueba de Lautaro Blanco
Sporting CP	-	193	

Jugadores de referencia (pid del parche): Haaland 133543 (reg 4340) - Neymar 40352 (reg 1936) - Guéhi 131286 - Stones 47785 - Mbappé 110718 - Isak 115079 - Lautaro Blanco 141350.

Competiciones dentro de la Liga Máster (listas de equipos, bloques de 3000 B)

Lista	Equipos	Qué parece ser
0xB0D870	30	Liga argentina (Primera)

0xB0E428	16	Liga ecuatoriana
0xB0EFE0	20	Liga colombiana
0xB0FB98	16	Liga chilena
0xB10750	20	Brasileirão
0xB11308	18	Liga 1 Perú
0xB11EC0	20	LaLiga
0xB12A78	20	Premier League (la del usuario; ID de competición 9)
0xB13630	20	Serie A
0xB141E8	16	Chile segunda (Cobreloa...)
0xB14DA0	18	Ligue 1
0xB15958 / 0xB17C80	12	Costa Rica (dos listas)
0xB193F0 / 0xB19FA8	47 / 44	Copas sudamericanas (Sudamericana / Libertadores, fases)
0xB1B718	19	Champions (clubes europeos)
0xB1E5F8	25	Concacaf
0xB1F1B0	66	Argentina ampliada (Primera + Nacional)
0xB1FD68 / 0xB20920 / 0xB214D8 / 0xB22090 / 0xB22C48	36 / 40 / 26 / 32 / 32	Colombia, Brasil, Ecuador, Chile y Perú ampliadas (con segundas)
0xB23800	18	Liga saudí
0xB243B8	28	MLS
0xB28A08 ... 0xB2B8E8	48 / 24 / 16 / 15 / 16	Selecciones (Mundial, Eurocopa, Concacaf, Asia, África)
0xB2D058 ... 0xB2F380	36 / 20 / 16 / 20	Segundas divisiones (Argentina, Brasil, Ecuador, Colombia)

IDs de competición vistos en el calendario del usuario: **9** = Premier - **63** = ventana de fichajes - 60, 62, 11, 7 = otros eventos.

Rarezas de este parche

- El equipo del usuario conserva un bloque de alineación «viejo» con el ID 173 (bloque 154) que el juego ya no actualiza.
- Las tablas C y D del usuario tienen 32 juveniles (regs >= 16358) entre el primer equipo y los fichados.
- Banca de 12 en la carrera auditada.
- La primera subida del catálogo a la web falló por un jugador sin nombre (ya saneado).

Pruebas hechas en el juego con este parche

Ver ../PRUEBAS.md: 13 pruebas (10 [PROBADO], 3 [NO] corregidas). Traspasos probados: Haaland City->Santos, Mbappé Madrid->Santos, Guéhi City->Madrid, Isak Liverpool->Santos; presupuesto 500 M / tope 200 M.

Auditoría de la carpeta del parche (8 oct, 16:50) - [OBSERVADO] OBSERVADO

Dónde: D:\Frank\Games_\Conmegol Patch\ (es la carpeta del juego con el parche encima). conmegol_info.txt: modo «ConmeGOL Patch 26», Steam ID 239200, save en Documentos\KONAMI\Football PES 2021 SEASON UPDATE\239200\save.

download\ (lo que carga el juego por DpFileList.bin): dt80_100E...700E_x64.cpk (7 CPK oficiales de Konami, 1,0-2,3 GB cada uno, 31 oct 2025) + **CGP_database.cpk (3,7 MB)**, la base del parche, cargada al final (pisa a los demás). La carpeta Data\ tiene los CPK base del juego (dt00...dt14). Hay además ConmeGol Extras\ con tres variantes del parche («Clasicos (No LM)», «ConmeGOL Patch 26», «...(Peru, Chile, Uruguay B)»), cada una con su download\, SiderAddons\ y CPY.ini: el instalador copia una u otra.

Formato de DpFileList.bin (400 B, 8 entradas): cabecera 16 B [0x64][n entradas u32][0x64][0x2774] y luego **48 B por CPK:** [orden u32, de mayor a menor: 8...1][id u32][id + 0x2710 u32][nombre ASCII, 32 B con ceros]. IDs vistos:

dt80_100E->0x64..., CGP_database -> orden 1, id 0x190. Es fácil de generar desde el programa ([HIPÓTESIS] falta probar uno generado por nosotros).

Sider (SiderAddons\, sider 7.x del 19 ene 2025): livecpk.enabled = 1 con **más de 60 cpk.root** apuntando a carpetas sueltas en livecpk\ (Faces, Kits, Logos, Menu, Scoreboard, Stadium..., PRDX Body Model con decenas de variantes por marca) y **~70 módulos Lua** (lua.module): kserv (uniformes), StadiumServer, BallServer, scoreboard-hexx + CGP_ScoreboardServer, Entrance, Chants/Commentary/ Sound servers, cámaras, UIColors, MenuServer, MiniFaceServer, Competition-Server, **MasterLeague.lua** (selector de sala de prensa de la LM), **StartingYearChanger.lua** (año de inicio de LM/BAL: busca en memoria 8B 70 48 48 8D 96 1E 2A 64 01 B8 <año u16> 00 00 66 y lo cambia), **Chatty_AutoLineup** (alineación automática nativa), real-coach-team, Real-Time-Match-Manager, attack_mentality...

Conclusión: **ConmeGOL es, en su mayor parte, un parche «Sider»**: la base del juego sigue siendo la de Konami y casi todo lo visual entra por livecpk sin reempaquetar CPK. Para el parche Phoenix esto es la vía rápida: base propia (.cpk pequeño como CGP_database.cpk) + contenido suelto por LiveCPK + módulos Lua propios.

Pista fuerte para el calendario (módulos FWC_Calendario / FWC_Grupos / FWC_Llaves, bytecode LuaJIT con cadenas legibles): escriben en MEMORIA el fixture de un torneo. Formato de cada partido (20 B): [id u16][id u16][0][comp/ronda u16][? u16][año u16][mes u8][día u8] [...] [hora u16][?] (ej. 24 24 00 00 00 29 1C 00 20 EB 07 07 04 01 00 00 00 16 00 06 = partido 0x24, año 2027, 4 jul, 22 h). Ancla: 24 00 00 00 29 1C 00 60. Las rondas van como listas de IDs de partido (Round1 = 24 25 2A 2B 30 31 ...). Esto cuadra con los 10 IDs de partido por jornada que vimos en el calendario del usuario del guardado (§16): **el motor identifica los partidos por ID y guarda por jornada la lista de IDs; el emparejamiento se guarda aparte**. Dónde está en el guardado sigue pendiente (experimento antes/después).

La base de datos del parche por dentro (download\CGP_database.cpk, 3,7 MB) - [OBSERVADO] OBSERVADO (8 oct, 19:50)

Es el corazón del parche: define «el mundo» del juego. 17 archivos (los .bin de pesdb van con cabecera WESYS + zlib; Phoenix Mercado ya los extrae: cpk::listar, cpk::extraer, descomprimirWesys):

Archivo	Tamaño	Registros	Qué es (lo visto)
common/etc/pesdb/Player.bin	9.359.064 B	29.997 x 312 B	jugadores (ya decodificado: 01 §B)
PlayerAssignment.bin	326.368 B	20.398 x 16 B	plantillas de fábrica: [nº registro][jugador][equipo][dorsal u8 + banderas]
SpecialPlayerAssignment.bin	11.104 B	694 x 16 B	asignaciones especiales (¿leyendas/clásicos?) [HIPÓTESIS]
Team.bin	1.147.468 B	749 x 1.532 B	equipos (749 = los del option file)
Coach.bin	84.000 B	840 x 100 B	entrenadores
Stadium.bin	21.560 B	55 x 392 B	estadios
Competition.bin	3.240 B	90 x 36 B	competiciones: [0xc864][región][.][id][tipo][0][NOMBRE ASCII 28 B] (lista en datos/conmegol-26-competiciones-pesdb.json)
CompetitionEntry.bin	15.816 B	1.318 x 12 B	qué equipo juega qué competición
CompetitionRegulation.bin	493.920 B	[HIPÓTESIS]	reglas de cada competición (formato, n.º de equipos, fases) - clave para crear/agrandar ligas
Tactics.bin / TacticsFormation.bin	17.976 / 593.208 B	1.498 x 12 / 4.494 x 132	tácticas (2 por equipo) y formaciones (6 por equipo)
Derby.bin	3.348 B	279 x 12 B	clásicos/derbis
Country.bin	303.880 B	214 x 1.420 B	países
character/appearance/PlayerAppearance.bin	1.799.820 B	[HIPÓTESIS]	apariciencia (cara, cuerpo, accesorios)
boots/BootsList.bin, glove/GloveList.bin			botines y guantes

Competiciones que define el ConmeGOL 26 (90): Mundial de Clubes, Libertadores, Sudamericana, Recopa, Champions, Concacaf, ligas D1/D2 y copas de Argentina, Colombia, Brasil, Ecuador, Chile, Perú, Uruguay, Paraguay, Bolivia, Venezuela, Honduras, Costa Rica, México, MLS, España, Inglaterra, Italia, Francia, Arabia; selecciones (Mundial y eliminatorias, Euro, Copa América, Asia, África) y propias («CLASICO_EQUIPOS_CLUBES», «CLASICO_EQUIPOS_AMERICA», «LEYENDAS_DEL_FÚTBOL»).

Para el parche Phoenix: este .cpk es lo que hay que **fabricar** (equipos, plantillas, competiciones y sus reglas).

Pendiente: decodificar Team.bin, CompetitionEntry.bin y CompetitionRegulation.bin campo por campo, y saber **reempaquetar** el .cpk.

Parche: Sudamerican Project 2026 (PES 2021) - auditoría de carpeta hecha (8 oct); catálogo y Liga Máster pendientes

Qué	Dato
Nombre interno usado por Mercado	sudamerican-2026
Carpeta del parche	D:\SP2026\
Base de datos (CPK)	D:\SP2026\download\SP_Subs.cpk
Carpeta de guardados	...\KONAMI\Football PES 2021 SEASON UPDATE\292733975847239680\save
Option file	292733975847239680\save\EDIT00000000 (11.026.772 B cifrado)

Lo que ya se hizo con él: emparejamiento de jugadores ConmeGOL <-> Sudamerican (10.676 automáticos, 3.849 a revisar, 3.716 sin candidato; clubes 464 automáticos / 136 a revisar) y modo seguro cuando se mezclan option file y base de parches distintos.

Pendiente: catálogo propio, guardados de Liga Máster (ninguno auditado aún), comprobar que la estructura coincide con el motor (01-MOTOR-PES2021.md) y repetir las pruebas clave en el juego.

Auditoría de la carpeta (8 oct 2026, 20:45) - [OBSERVADO] OBSERVADO

Estilo de parche: «clásico» de CPK (lo contrario de ConmeGOL, que va casi todo por Sider/livecpk).

- download\DpFileList.bin: **34 CPK** (cabecera 64 / 0x22 / 64 / 0x2774, mismo formato que ConmeGOL). Orden: 7 dt80_* de Konami -> SP_Adboards_1...8 -> SP_Faces_0...12 (**13 CPK de caras de 5-11 GB cada uno**) -> SP_Music -> SP_Subs (la base) -> db_st_7.0_byJGames -> 3 narradores VIPER_* (Bambino Pons, Fernando Solabarrieta, Miguel Simón).
- En download\ hay CPK que **no** están en la lista (no se cargan): SP_Subs_WC.cpk (base del Mundial 2026) y PredsKanas2021_NatAnthems.cpk (himnos). Hay un DpFileList Generator dentro de la carpeta del juego.
- Dos Sider propios, cada uno con sus SP LAUNCHER <letra de disco>.bat: «**Sudamerican Project Sider**» (56 módulos, 107 cpk.root) y «**... WC26**» (Mundial, 66 líneas de módulo, 98 cpk.root).
- Mismos problemas que ya vimos en ConmeGOL (ver ../sider/AUDITORIA-SIDER.md): camera.lua + DynamicWideCam.lua juntos (pisan la misma instrucción), GFX_lod.lua, scoreboard-hexx. Además: **módulos cargados dos veces** (MenuServer en el normal; env, etrace, camera y netblock en el WC26), módulos de prueba activos (jittest, zlibtest, etrace) y **netblock.lua** (bloquea la red del juego: incompatible con cualquier online tipo PESBUL).
- Créditos (LEER.txt): más de 30 colaboradores de la comunidad (facemakers, Olmos Jr 23, Afandix...), Discord oficial.
- Mismo PES2021.exe que la carpeta de ConmeGOL (458.806.784 B, 16/4/2023, FileVersion 1.1.0.0).

Lección para Phoenix: CPK gigantes de caras dan un parche pesado y lento de actualizar; ConmeGOL demuestra que casi todo puede ir suelto por livecpk. Lo ideal: base de datos en un .cpk pequeño + resto por Sider, con **un solo** sider.ini limpio (sin duplicados, sin módulos de prueba, sin pares que se pisen).

Puente en vivo con Sider - riesgos y cómo se evitan

Este documento acompaña al módulo phoenix.lua (prueba 1, solo lectura) y a todo lo que venga después con Sider.
Regla de oro: **cada paso se prueba primero en la PC de Fralex, en partidos offline, y solo después con amigos.**

Qué se sabe del entorno (revisado el 2026-10-08 en la PC de Fralex)

- [OBSERVADO] Sider **7.3.3**; sider.ini pide 60 módulos Lua: 49 cargan y 11 no existen. Auditoría completa en AUDITORIA-SIDER.md.
- [OBSERVADO] El Lua de Sider (LuaJIT 5.1) trae: io (leer y escribir archivos), fs (find_files, make_dirs), memory (leer/escribir/buscar en la memoria del juego), ffi, zlib, match, audio, os (solo date, time, clock). **No trae red**: el juego no habla con Internet desde Lua (salvo por ffi, que no usaremos para eso).
- [OBSERVADO] El overlay de Sider se abre con **Espacio** (overlay.vkey.toggle = 0x20), se cambia de módulo con **1** y **%**. Solo se dibuja (y solo llama a los módulos) mientras está abierto.
- [OBSERVADO] Si un módulo de la lista no existe, Sider lo anota (PROBLEM: Unable to open file) y sigue: hoy hay 7 así en sider.ini y el juego funciona.
- [OBSERVADO] Hay **PESBUL** (PESBUL_2021_Server_Connector.bat): servidor comunitario que imita el online de Konami cambiando el archivo hosts de Windows. Es decir, el online "de verdad" ya lo da la comunidad; el puente Phoenix tiene que convivir con él.
- [OBSERVADO] Otros enganches en el mismo juego: ReShade (dxgi.dll), DLSS5 (addons de ReShade), PES21_Hook, CreamAPI. Cualquiera puede chocar con algo nuevo; por eso lo nuevo entra de a poco.

Cómo es el puente

Web Phoenix -> Phoenix Link / Phoenix Sync (PC) -> archivo avisos.txt -> phoenix.lua (dentro del juego) -> overlay

El juego no se conecta a nada: solo lee un archivo pequeño que la app de PC deja en SiderAddons\content\phoenix\.

Riesgos y medidas

#	Riesgo	Qué podría pasar	Medida tomada
1	Error dentro del módulo	Mensaje de error, o en el peor caso que el juego se cierre	Todo el trabajo va dentro de pcall; tras 5 errores seguidos el módulo se apaga solo y lo anota en sider.log. Prueba 1 no usa memory ni hooks del partido.
2	Archivo leído a medio escribir	Texto cortado o raro	La app escribe en avisos.tmp y luego reemplaza avisos.txt de una vez (con reintentos si el juego lo está leyendo).
3	Archivo gigante o con basura	Lentitud o caracteres extraños	Se leen como máximo 4 KB, 14 líneas, 110 bytes por línea; si no es UTF-8 válido no se muestra; se quitan caracteres de control.
4	Bajón de rendimiento (FPS)	Tirones	Solo trabaja con el overlay abierto y mira el archivo cada 2 s como mucho. Con el overlay cerrado su coste es cero.
5	Choque de teclas con otros módulos	Una tecla hace dos cosas	La prueba 1 no usa ninguna tecla propia (MasterLeague usa 0, 6-9, RePág/AvPág, Supr; BallServer, F3...).
6b	Instalar en la carpeta equivocada	El switcher pisa el cambio (pasó el 8 oct: la línea se perdió y el módulo nunca cargó)	Instalar en ConmeGol Extras\ConmeGOL Patch 26\SiderAddons, que es la que copia el switcher.

6	Romper sider.ini	Sider no carga ningún mod	Antes de tocarlo se guarda sider.ini.respaldo-phoenix; el cambio es una sola línea al final de la lista de módulos. Desinstalar = quitar esa línea o restaurar el respaldo.
7	Una actualización de ConmeGOL pisa sider.ini	El módulo deja de cargarse (no rompe nada)	Documentado: hay que volver a añadir la línea. El instalador de Phoenix lo comprobará en el futuro.
8	Online con PESBUL / partidas entre dos PCs	Desincronización si los dos juegos no tienen lo mismo	La prueba 1 solo dibuja texto: no cambia nada del partido, así que no puede desincronizar. Para módulos que escriban memoria: apagados en online y probados solo offline hasta demostrar que son seguros.
9	Jugar con amigos	Que algo se rompa en sus PCs	Con Phoenix Link (Parsec) el juego corre solo en la PC del anfitrión: los amigos no instalan nada. Es el modo recomendado para las pruebas.
10	Escribir memoria en sitios equivocados (fases futuras)	El juego se cierra o guarda datos malos	Antes de escribir: localizar por patrón (AOB) y comprobar que lo leído tiene sentido (como hacen StartingYearChanger y Chatty_AutoLineup); nunca escribir con el juego guardando; probar cada valor en una partida desechable; guardados de respaldo.
11	Juego trampa / ventaja injusta	Desconfianza en la liga	Lo que cambie el partido (dinero, plantillas, reglas) lo decide la web (firmado con Ed25519, como /liga/cambios), nunca el jugador.
12	Archivos con nombre raro / rutas	No encuentra el archivo	La ruta se arma desde ctx.sider_dir (la que Sider da); la app comprueba que exista sider.ini antes de escribir.

Fases

- **Prueba 1 (ahora): leer archivo -> overlay.** No toca memoria. Objetivo: comprobar que el puente funciona y que el juego sigue estable.
 - **Prueba 2:** Phoenix Link escribe los avisos solo (resultado de la sala, fichajes confirmados por la web).
 - **Prueba 3:** leer (no escribir) datos del partido con match / ctx (marcador, equipos) y dejarlos en un archivo para que la app los suba a la web.
 - **Prueba 4:** escribir **un** valor conocido en memoria (p. ej. dinero de la Liga Máster), solo offline, con verificación.
- Cada prueba se anota en PhoenixSync/PRUEBAS.md.

Auditoría de Sider en la PC de Fralex (8 oct 2026)

Antes de instalar cualquier módulo Phoenix, se leyó **cada módulo activo** de Sider: qué hace, qué memoria del juego toca, qué archivos cambia, qué teclas usa y con quién choca. Detalle completo (módulo por módulo, con direcciones y bytes) en auditoria/: hallazgos_A_shirtless.md, hallazgos_B_memoria.md, hallazgos_C_camaras.md, hallazgos_D_archivos.md e inventario-modulos.json.

Límite importante: el sider.log revisado es de una sesión **sin partido** (solo menús, cierre limpio). Lo que pasa durante un partido se dedujo leyendo el código, no viéndolo. Falta un sider.log de una sesión con partido jugado.

Cómo está montado

El **switcher** de Fralex, al elegir «ConmeGOL Patch 26», **copia** ConmeGol Extras\ConmeGOL Patch 26\SiderAddons (sider.ini, modules, content, livecpk, olmosjr23) **encima** de SiderAddons del juego. Lo que no está en la carpeta del parche se queda.

- Consecuencia: el sider.ini que manda es el de **la carpeta del parche**. Un cambio solo en SiderAddons del juego se pierde (así se perdió la línea de phoenix.lua, que por eso nunca se activó).
- La carpeta del parche trae 23 archivos de módulos; los otros 63 están solo en la del juego (añadidos a mano o de versiones viejas).
- Sider **7.3.3**. sider.ini pide **60 módulos: 49 cargan, 11 no existen** (tunnel, Stadium_Banner, Stadium_Board, Stadium_CornerFlag, GoalSongServer, Derbys, cr7_audio, BallBoysServer, tournament_anth_tunnel, Nets-Server - el archivo real se llama nets.lua -, CornerFlag). Sider los salta sin romper nada.
- Cada módulo corre en **su propio entorno** Lua: los nombres globales repetidos (init, home, away...) **no** chocan (comprobado con el volcado de env.lua). Lo único compartido: la tabla string, math.randomseed y las declaraciones ffi.cdef (VirtualAlloc, compatible).
- Teclas: casi todos usan 0, 6-9, -, +, RePág/AvPág y el stick derecho, pero Sider 7 solo manda las teclas al módulo **elegido en el overlay**, así que no chocan (supuesto respaldado por el log).

Problemas encontrados (ya existían, no los puso Phoenix)

#	Gravedad	Qué	Por qué importa	Arreglo propuesto
1	[ROJO] Alta	camera.lua y DynamicWideCam.lua parchean la MISMA instrucción (0x1408a1fbe). Cada uno suma +0x2C: la cámara Dynamic Wide termina leyendo 0x1425985d0 y DynamicWideCam escribe 0.1 ahí, encima de bytes que eran texto (6E 64 00 00) - comprobado en el log (líneas 4468-4470 y 8020-8025).	Corrompe datos del juego; cada Shift+R lo empeora. Candidato serio a cierres raros.	Quitar camera.lua (lo único propio que aporta es activar/desactivar repeticiones).
2	[ROJO] Alta	GFX_lod.lua escribe en una dirección fija (0x14297B0C1) pensada para otra versión del exe, sin comprobar qué hay ahí.	Escritura a ciegas.	Quitarlo o reescribirlo con búsqueda por patrón.
3	[NARANJA] Media-alta	SoundServer.lua sigue 4 punteros desde una dirección fija sin comprobar nulos y escribe el volumen cada fotograma con su overlay abierto.	Puede leer/escribir en la nada.	Revisar o quitar.
4	[NARANJA] Media	CGP_SB_Addons_2.lua (bytecode del parche) escribe colores del marcador en memoria cada fotograma, sin pcall, con desplazamientos fijos a un exe.	Del propio ConmeGOL; se desactiva si no encuentra la firma.	Dejarlo (es del parche) y vigilarlo en el log.

5	[NARANJA] Media	shirtless_celebration.lua (añadido a mano): 15 parches de código + 512 KB de memoria ejecutable; valida antes de escribir y se apaga si no coincide, sin pcall; copia 127 archivos (32,8 MB) en cada arranque y escribe un log línea a línea; redirige 17 modelos sin comprobar que existan.	Posibles tirones; cierre si faltan esos modelos al celebrar.	Comprobar los 17 archivos; apagar su log de depuración.
6	[NARANJA] Media	real-coach-team.lua fabrica un .fpk en disco al empezar cada partido y mira el disco 2 veces por cada archivo que carga el juego.	Lentitud en cargas; si falla el .fpk, banquillo raro.	Vigilar.
7	[NARANJA] Media	SleeveBadge, Commentary-Server, Competition-Server: punteros sin comprobar / patrón débil / sin validar. Commentary usa el idioma 0x18 (fuera de rango) como respaldo.	Riesgo bajo pero real.	Vigilar en el log con partido.
8	[AMARILLO] Baja	Duplicados y conflictos de archivos: BroadCastCam y StadiumCam tocan el mismo valor (gana StadiumCam); FanViewCam y camera.lua igual; Entrance_fix gana a Entrance; IntroServer y movieintro renombran el mismo vídeo de intro (si el .usm no existe -> pantalla negra).	Cosmético, salvo la intro.	Elegir uno de cada par.
9	[AMARILLO] Baja	Módulos que no hacen nada: Chatty_AutoLineup (es para Football Life 2026, se negó a cargar), scoreboard-hexx (solo para un marcador EPL que no se usa), goalscreams (pide Sider 7.3.4+), Anti-Cheat (placebo: inventa campos que Sider no tiene; además está desactivado), env.lua (solo vuelca datos al log), matchset.	Ruido, carga inútil.	Quitar.
10	[AMARILLO] Baja	sider.ini: «Menu Light» dos veces, un tabulador suelto, 11 módulos inexistentes; faltan modules\IMAGENS\gervol.png y chantssserver.png (SoundServer, Chants-Server); UIColors no encuentra topModeSelectDMM.bin.	Cosmético.	Limpiar.
[OJO]	Desactivado	attack_mentality.lua (comentado) es de alto riesgo si se vuelve a activar: reescribe código del juego mientras se ejecuta.	-	No activarlo.

Otros: ReShade.log tiene errores de compilación de algunos shaders (no afectan al juego); dlss5-feed-crash.dmp está vacío (0 B, 8 oct 22:10): no da información.

Lo que esto significa para Phoenix

- **Ninguna** de las zonas de memoria que tocan los módulos actuales se cruza con lo que hace phoenix.lua: la prueba 1 **solo** usa el evento overlay_on (dibujar texto) y lee un archivo propio. No escribe memoria, no redirige archivos, no cambia ctx, no usa teclas. Es la superficie de choque más pequeña posible.
- **Certeza:** nadie puede dar un 100 % matemático con 49 módulos de terceros y un juego cerrado. Lo que sí se

puede es: (1) no compartir nada con ellos (hecho por diseño), (2) arreglar los choques que ya existen (tabla), (3) probar de forma controlada - partido sin el módulo, partido con el módulo, comparar los dos sider.log - y (4) que el módulo se apague solo ante cualquier error.

- **Para módulos futuros:** IntroServer.lua **cambia ctx.tournament_id** (86/87->88, 89->90): un módulo Phoenix que lea la competición verá el valor cambiado. Los módulos CGP comprueban que ciertos equipos estén en ciertas competiciones (111, 128, 137, 90): Phoenix Mercado mueve **jugadores**, no equipos, así que no les afecta; si algún día mueve equipos entre competiciones, hay que respetarlo.
- **Dónde instalar:** en la carpeta del parche (ConmeGol Extras\ConmeGOL Patch 26\SiderAddons), para que el switcher lo lleve.

Plan propuesto (cada paso con permiso de Fralex)

- Jugar **un partido** con el Sider actual (sin Phoenix) y guardar ese sider.log como referencia.
- Limpieza con copia de seguridad: quitar camera.lua, GFX_lod.lua y los módulos que no hacen nada; quitar las 11 líneas de módulos inexistentes y el «Menu Light» repetido. Otro partido y comparar logs.
- Recién entonces, añadir phoenix.lua (una línea, en la carpeta del parche) y otro partido de prueba.

¿Se puede cambiar el juego en tiempo real desde la web? - análisis (2026-10-09)

Cómo funciona el switcher de ConmeGOL («PES2021 Start.exe»)

- Programa en Python 3.14 empaquetado con PyInstaller y hecho con Gemini (script gemini-code-1785843135404.pyw, interfaz tkinter).
- Opciones: ConmeGOL Patch 26 - Latam (PE, CL, UY B) - Clásicos (No LM).

Al elegir una opción:

- ejecuta robocopy "ConmeGol Extras\<modo>" "<carpeta del juego>" /E /IS /IT, que **copia y sobrescribe todo** (CPY.ini, SiderAddons, conmegol_info.txt, download);
- escribe el nombre del modo en version_actual.txt;
- abre SiderAddons\sider.exe, espera 2 segundos y abre PES2021.exe.
- **No usa internet:** no actualiza nada ni se conecta a ningún sitio.
- Consecuencia: lo que pongamos en la raíz se borra al volver a usar el switcher. Lo nuestro debe vivir dentro de ConmeGol Extras\<modo>\SiderAddons, o en un modo propio «Phoenix».

Lo que permite Sider en este parche (sider.dll, LuaJIT 2.1)

- **Eventos:** display_frame (se llama en cada cuadro de imagen), key_down, gamepad_input, overlay_on, set_teams, set_conditions, set_match_settings, set_match_time, livecpk_get_filepath, livecpk_data_ready, livecpk_rewrite, trophy_rewrite y otros.
- **Librerías:** io (leer y escribir archivos), memory (leer y escribir la memoria del juego), ffi (llamar funciones de Windows).
- **Sin internet:** no trae nada de red.
- **Precedente en la carpeta:** Real-Time-Match-Manager.lua (de heazebell) lee un .ini y cambia el clima en vivo por memoria. Ojo: está hecho para FL_2026.exe y Sider 7.4 y está desactivado en sider.ini; su autor dejó en su lista negra require(ffi).
- **Recarga en vivo:** Shift+R recarga los módulos Lua sin cerrar el juego (vkey.reload).

Arquitectura viable

Web (Supabase) <-> **Phoenix Link/Mercado** (programa en la PC, hace todo lo de internet) <-> **buzón** (archivos en content\phoenix\) <-> **módulo Sider phoenix.lua**, que revisa el buzón 1 vez por segundo dentro de display_frame, sin red y sin ffi.

Qué se puede cambiar y cuándo se ve

Qué	Cómo	Cuándo se ve	Riesgo
Avisos de la web en pantalla	buzón -> overlay	al momento	muy bajo
Resultados del partido -> web	match.stats() -> archivo -> Link lo sube	al terminar el partido	bajo
Rival, estadio, clima, hora y reglas del próximo partido	set_teams, set_conditions, set_match_settings	en el siguiente partido	bajo (los módulos de ConmeGOL ya lo hacen)
Caras, uniformes, escudos, botines, cánticos	livecpk_get_filepath -> archivo nuevo	la próxima vez que el juego cargue esa imagen (menú o partido)	bajo
Habilidades, posición, dorsal y altura de un jugador	livecpk_data_ready parchea Player.bin mientras el juego lo carga	al reiniciar el juego	bajo

Lo mismo, pero sin reiniciar	escribir en la tabla de jugadores en memoria	en el menú, antes del partido	medio-alto: hay que mapear la memoria; solo en menús, nunca a mitad de partido
Dinero, contratos y fichajes de la Liga Máster	Phoenix Sync edita la partida guardada con el juego cerrado	al cargar la partida	bajo (ya probado)
Lo mismo en memoria, con el juego abierto	memoria	-	alto: no se hace

Reglas de seguridad

- Nada de internet dentro del juego: el programa de la PC hace la red y Sider solo lee archivos pequeños.
- Escritura atómica del buzón (ya hecha en PhoenixAviso.ps1).
- Todo dentro de pcall; si hay 5 errores seguidos, el módulo se apaga solo.
- Ninguna escritura en memoria durante un partido.
- Instalar en la carpeta del modo, no en la raíz.

Phoenix Mercado - Diario de pruebas (prueba y error)

Para qué sirve: dejar anotado cada intento: qué se probó, qué se vio en el juego, qué falló, por qué y cómo se corrigió. Así se puede ver el avance y, cuando todo esté probado, convertirlo en el **PDF final**.

Reglas: solo se **añade** (no se borra lo que salió mal: es parte del aprendizaje). Lo escriben las dos cuentas de Claude. Detalle técnico completo: liga-master/ESTRUCTURA-ML.md (estructura del archivo) y REGISTRO.md (bitácora de cambios).

Cómo leerlo: [PROBADO] salió bien - [NO] hubo un error - [PENDIENTE] pendiente - [OBSERVADO] encontrado revisando los archivos (sin verlo aún en el juego)

Ranuras: en el menú Cargar de la Liga Máster, la ranura N es el archivo ML0000000(N-1). Ejemplo: ranura 9 = ML00000008.

Resumen (al 8 oct 2026, 13:40)

- **Pruebas mirando el juego:** 13 - [PROBADO] 10 - [NO] 3 (las 3 ya corregidas y vueltas a probar)
 - **Otros errores:** subida del catálogo a la web [NO] (corregido; [PENDIENTE] falta volver a ejecutar) - alineación del option file en amistosos [OBSERVADO] ([PENDIENTE] por corregir) - venta bloqueada después de jugar un partido [OBSERVADO] ([PROBADO] corregido) - contratos dobles [OBSERVADO] ([PENDIENTE] prueba pedida)
 - **Pruebas automáticas:** 117 de 117 bien (en Linux) - compilación en Windows [PENDIENTE]
-

Parte 1 - Option file (EDIT00000000: lo que usan los amistosos y las salas)

7 oct - Leer y volver a guardar el option file - [PROBADO] Se descifra y se vuelve a cifrar, y queda **idéntico** al original. 740 equipos leídos.

7-8 oct - Mover a Lautaro Blanco (Boca -> Sporting) en una copia - [PROBADO] en lo que se miró Se vio en el juego en el Sporting **con su cara**. > [OBSERVADO] **8 oct (13:00), revisando los archivos:** la **alineación de Boca se corrió un puesto**. En un amistoso, Boca habría > salido con el portero **Marchesín de delantero** y cada jugador, desde el lateral izquierdo, un puesto más atrás; además el > **capitán habría cambiado**. El Sporting estaba bien. Causa: cada equipo del option file también tiene una **lista de > alineación** (orden de titulares + roles) y el programa no la tocaba. Es el mismo tipo de error que el de la Liga Máster > (ver [NO] de la ranura 8). [PENDIENTE] Se corrige antes de seguir y se prueba en un amistoso.

8 oct - Modo seguro - [PROBADO] Si el option file y la base de datos son de parches distintos, el programa se pone en modo seguro y no toca nada.

8 oct - 49 pruebas en la PC de Fralex - [PROBADO] 49 de 49 Conexión con la web, cuenta, cifrado, catálogo, emparejamiento, copias de seguridad.

8 oct (09:13) - Subir el catálogo a la web (SUBIR.bat) - [NO] La web rechazó el primer lote: había un jugador **sin nombre**. Se corrigió el catálogo (commit 666a2b3). [PENDIENTE] Falta volver a ejecutar SUBIR.bat.

Parte 2 - Liga Máster interna (tu carrera guardada)

Ranura 1 - v2 - Haaland City -> Santos tocando solo la lista del club - [NO] En la búsqueda Haaland quedaba **sin equipo**, pero seguía en la Estrategia y en la zona de ventas del City. Aprendido: tu equipo guarda **más listas por jugador** (tablas, alineación) que hay que tocar todas a la vez.

Ranura 2 - Copia limpia de control - [PROBADO] Sirve para comparar: Haaland en el City, todo normal.

Ranura 3 - Mbappé Madrid -> Santos (dos equipos de la IA, solo la lista del club) - [PROBADO] en lo que se miró Mbappé aparecía bien en el Santos. > [OBSERVADO] Después se supo (ranura 8) que la **alineación** del Madrid y del Santos tampoco se había tocado. Esa pantalla no se miró.

Ranura 4 - Haaland City -> Santos, solo listas - [PROBADO] a medias Aparecía en el Santos; el lado del City no se revisó.

Ranura 5 - v5 - Haaland City -> Santos completo (listas + tablas + lista K) - [NO] Haaland perfecto en el Santos, pero en la **Estrategia del City aparecía Rulli (portero) en el ataque**. Causa: la alineación y los 6 roles (capitán, lanzadores)

son **números de puesto en la plantilla**; al sacar a Haaland esos números se corrían un lugar.

Ranura 6 - v6 - Igual, recalculando alineación y roles (Neymar en lugar de Haaland) - [PROBADO] Estrategia bien (Neymar delantero, ningún portero arriba), City con 25, Haaland en el Santos. Después Fralex **jugó un partido y guardó en la ranura 7** - [PROBADO] el juego no corrigió nada por su cuenta. > Nota: al guardar, el juego **sobrescribió** el archivo de la ranura 6. Lección: anotar la huella (md5) de cada archivo > que se entrega, para saber después si es el nuestro o uno guardado por el juego.

Ranura 8 - v7 - Primera prueba del programa en C++: Guéhi City -> Real Madrid (Stones titular) - [NO] [PROBADO] Pantalla Equipos (Madrid 31 con Guéhi, City 24), búsqueda y Estrategia del City. [NO] En la **Alineación del Madrid** salía un **jugador en blanco «DC 0»** en lugar de Guéhi. Causa: **cada equipo de la IA tiene su propia lista de alineación** (bloque de 600 B) y no se actualizaba.

Ranura 9 - v8 - Arreglo de la alineación de la IA + Isak Liverpool -> Santos - [PROBADO] - Madrid: Guéhi al final de la lista, con el 30, sin jugador en blanco. - Liverpool: **Ekitiké** (elegido solo, por posición) juega en el puesto de Isak. - Santos: Isak al final de la lista, con el 99. - City: sin cambios, Stones titular. > Hallazgo: en esta partida la **banca es de 12**, no de 7. El programa ya no supone un tamaño fijo. > [PROBADO] 8 oct 13:07: Fralex jugó un partido y guardó en la **ranura 10**: todo se mantuvo (ver abajo).

Parte 3 - Pruebas automáticas (las corre la computadora sola)

Cuándo	Resultado	Qué cubren
8 oct, mañana	41 de 41 [PROBADO]	Option file, web, copias, catálogo
8 oct 11:35	41 de 41 [PROBADO]	Lo mismo + el módulo de Liga Máster compilando
8 oct 11:55	88 de 88 [PROBADO]	Venta de tu equipo a la IA (datos de prueba)
8 oct 12:36	112 de 112 [PROBADO]	Alineaciones de la IA, traspasos IA <-> IA, sustitutos
8 oct 13:00	117 de 117 [PROBADO]	Sustitutos sin suponer el tamaño de la banca
8 oct 13:35	119 de 119 [PROBADO]	La venta ignora la ficha del último partido
8 oct 21:30	211 de 211 [PROBADO] (sin archivos) - 295 de 295 [PROBADO] (con option file, base, catálogo y guardado real)	Fichaje para el usuario desde la sincronización con la web: da exactamente los mismos bytes que el fichaje validado en el juego (prueba 19); condiciones opcionales de la web (monto, sueldo, cláusula, fin de contrato, dorsal)
-	[PENDIENTE]	Compilar en Windows (COMPILAR_MERCADO.bat)

Además, en archivos reales: el programa en C++ da **exactamente el mismo archivo** que el prototipo que el juego aceptó (ranura 6) y vuelve a generar la ranura 9 idéntica (misma huella).

Parte 4 - Lecciones aprendidas

- Mover a un jugador **solo en la lista del club no basta**: hay otras listas que dependen de ella.
- La alineación y los roles son **números de puesto**: si alguien se va, hay que reacomodarlos.
- **Los equipos de la IA también tienen alineación**, y en el option file también (amistosos).
- No suponer tamaños fijos: la **banca** puede ser de 7 o de 12.
- Cada prueba se hace en un **archivo nuevo** y se anota su huella.
- Mirar **todas** las pantallas del jugador movido y de **los dos equipos**: Equipos, Alineación/Estrategia y búsqueda.

Registro de nuevas pruebas (se añade abajo)

8 oct 13:07 - Ranura 9 -> jugar un partido -> guardar en la ranura 10 - [PROBADO] Todo se mantuvo: Guéhi en el Madrid, Isak en el Santos, Ekitiké en el Liverpool, Stones titular del City. El técnico de la IA puso a **Isak de titular** en el Santos por su cuenta (Haaland pasó a la banca). Todas las alineaciones de la IA siguen bien. > Ojo: el juego **guardó solo** encima de la ranura 9 al terminar el partido. Por eso cada prueba se identifica por su huella.

8 oct 13:20 - Revisión del guardado de la ranura 10 (después del partido) - [OBSERVADO] [NO] -> [PROBADO] AI

intentar vender con ese archivo, el programa **se negaba** («tabla dañada»). No rompía nada (se frenó solo), pero no dejaba vender. Causa: al jugar, el juego crea una **ficha del partido** con los que jugaron; empieza igual que una lista de la plantilla y el programa la confundía. Arreglo: solo cuenta como lista de la plantilla la que tiene a **todos** los jugadores; la ficha del partido no se toca. Probado con tu archivo real: venta de Max Alleyne (reserva) y de Stones (titular, con Aké de sustituto), todo verificado.

8 oct 13:30 - Contratos dobles - [OBSERVADO] [PENDIENTE] La tabla I guarda los **contratos**. Tres jugadores del City tienen **dos** (Rulli, Bettinelli y Stones; el segundo de Stones dura hasta 2028). El programa solo borra el primero al vender, así que el segundo quedaría como un contrato «fantasma». Para hacerlo exactamente como el juego: **rescindir el contrato de Stones en el juego** (ranura 10), guardar en una ranura nueva y comparar qué borra el juego. Mientras tanto, el archivo de prueba «v9 Stones al Madrid» **no se entrega**.

8 oct 13:30 - Ranura 10 -> «Despedir» a Stones -> guardar en la ranura 11 - [PROBADO] (prueba de referencia) Stones quedó libre y el City con 23. Ahora sabemos qué borra **el propio juego** cuando un jugador se va: **sus dos contratos** (el segundo era una oferta de otro club) y **la negociación abierta**. Nuestro programa solo borraba el primer contrato y no tocaba la oferta -> [PENDIENTE] hay que agregarlo antes de dar la venta por terminada. Detalle en ESTRUCTURA-ML §12. > Dato: la ranura 11 se llama ML0000000A (el juego cuenta en hexadecimal).

8 oct 13:55 - Escaneo de dinero y competiciones (sin probar aún en el juego) - [OBSERVADO] Se encontró dónde están el **presupuesto de fichajes** y el **tope salarial** de tu club, y cómo se calcula el presupuesto salarial (tope - sueldos). Los **sueldos y cláusulas** de cada jugador están en la tabla de contratos. Los clubes de la IA **no tienen dinero** guardado. Las **ligas de la Liga Máster** tienen su lista de equipos con espacio de sobra. Detalle en ESTRUCTURA-ML §13.

8 oct 14:10 - Ranura 12 (ML0000000B) - Presupuesto de fichajes 500 M y tope salarial 200 M - [PROBADO] En el juego: **Ppto. fichajes 500.000.000 EUR** y **Ppto. salarial 86.096.000 EUR** (= 200.000.000 - 113.904.000, la cifra exacta que se había calculado). El dinero del club se puede fijar desde fuera.

8 oct 14:10 - El «blob» era zlib - [OBSERVADO] La zona de 1,2 MB que nadie entendía se descomprime (4,7 MB) y guarda el calendario y los resultados de todas las ligas. Detalle §14.

8 oct 14:30 - Dentro del blob: la ficha de Liga Máster de cada jugador - [OBSERVADO] 16.422 fichas de 156 B con sueldo, valor de mercado (Días 55 M = pantalla OK), contrato, competiciones inscritas, valoración, crecimiento y condición. Ver ESTRUCTURA-ML §15. El calendario de partidos todavía no se localizó.

8 oct 15:00 - Búsqueda del calendario de partidos - [OBSERVADO] [NO] (todavía) Se probaron todas las formas habituales de guardar un calendario y ninguna aparece. Pista fuerte: cada jornada tiene **10 números de partido** (ej. 285-294) en el calendario del usuario. Falta un experimento: guardar justo antes y justo después de jugar un partido, el mismo día. Ver ESTRUCTURA-ML §16.

8 oct 15:35 - Laboratorio: la venta del C++ comparada byte a byte con el despido de Stones hecho por el juego - [PROBADO] Se reprogramó la venta para hacer exactamente lo que hace el juego (huecos en A-F/J/M, compactar G/H/contratos, borrar todas las entradas de contratos y la negociación abierta) y se comparó el resultado sobre la ranura 10 con la ranura 11 del juego: **todas las tablas, los contratos y las negociaciones quedan idénticos** (solo difieren bytes de relleno que el juego limpia, y lo que no replicamos aún: noticias, caja, medias del equipo). Antes de esto, la venta dejaba un contrato fantasma y la oferta abierta. Detalle en ESTRUCTURA-ML §17. 146/146 pruebas sintéticas.

8 oct 15:40 - Prueba 14 - Ranura 13 (ML0000000C) - Bettinelli City -> Lanús (dorsal 99), desde la ranura 11 del juego - [PENDIENTE] EN PRUEBA Primera venta con la lógica nueva y partiendo de un guardado en el que el juego aún no había rehecho la lista K. Qué mirar: plantilla del City (22), negociaciones (sin la de Lanús; sigue la del Deportivo por Rulli), contratos (sin Bettinelli; ppto. salarial +-2,3 M), Lanús con Bettinelli al final, ficha de Guéhi en el Madrid (club/contrato), jugar y guardar.

8 oct 16:20 - Prueba 15 - Option file EDIT_prueba_boca (md5 05bb20fe...) - Lautaro Blanco (LB titular de Boca) -> club 193, Braidá de LB - [PENDIENTE] EN PRUEBA Primera prueba de la corrección de alineación en el option file (bloque de tácticas, core/Alineacion.h). Antes, al mover a Blanco la formación de Boca se corría un puesto (Paredes de LB ... Marchesín de delantero). Qué mirar en un amistoso con Boca: Montero de arquero, Braidá de LB, el resto del once igual, Marchesín en el banco. Archivo hecho desde save\resplado\EDIT00000000 (md5 027b0fa2...).

8 oct 17:05 - Resultado prueba 14 - Ranura 13 (ML0000000C) - [PROBADO] En el juego: **Bettinelli en CA Lanús** (último de la plantilla, 29/29; ficha: equipo CA Lanús, liga Torneo Apertura), y el **Manchester City** con su alineación 4-1-3-2 completa y coherente (Donnarumma de arquero y capitán, sin huecos ni jugadores en blanco; Rulli encabeza la banca). Primera venta con la lógica «igual que el juego» (contratos, ofertas, negociaciones, huecos) confirmada en pantalla. Pendiente de esta ranura: jugar/guardar y el fichaje (ranuras 14/15).

8 oct 17:20 - Prueba 16 (referencia del juego) - Fichaje de Yann Sommer (Inter -> City, dorsal 12) desde la ranura

2, guardado en la 15 - [OBSERVADO] Fralex fichó en el juego (el mercado de la ranura 13 ya estaba cerrado el 31/8, por eso se usó una ranura de inicio de temporada). Se comparó con la base (= respaldo r0): el juego añade a Sommer en el primer hueco de cada tabla del usuario, con datos propios del jugador (curva de crecimiento, estadísticas, contrato, historial, dinero). Detalle en ESTRUCTURA-ML §19. Ojo: el juego guardó también encima de la ranura 2.

8 oct 18:32 - Resultado prueba 15 - Option file EDIT_prueba_boca (regenerado, md5 f91c6c45...) en un amistoso Boca-Huracán - [PROBADO] Alineación de Boca en la pantalla de estrategia: **Montero** de arquero, **Braida (Malcom) de LI** en el puesto de Blanco, el resto del once igual (Di Lollo, Costa, Lozano, Paredes, Delgado, Villa, Aranda, Merentiel, Valencia) y **Marchesín primero en el banco**. El error del portero suplente de delantero quedó corregido: el bloque de tácticas del option file se actualiza bien.

8 oct 18:41 - PROBAR2.bat en la PC de Fralex (ejecutables cruzados con MinGW) - [PROBADO] Conexión con la web OK, cuenta OK (token de Phoenix Link), **271/271 pruebas** con el option file real y el catálogo, y la **sincronización real con la web**: GET /liga/cambios?desde=0 respondió, **la firma Ed25519 se verificó** con la clave incrustada y la liga está en la versión 0 (sin traspasos aún). El original no cambió (huella igual). El paso 5 (modo código manager) dio NO_VINCULADO, que es lo esperado: esa PC usa el token compartido.

8 oct 19:05 - Prueba 17 - Ranura 1 (ML00000000, md5 92c9e6b2...) - Reescritura del blob comprimido - [PENDIENTE] EN PRUEBA Hecha desde la ranura 15 del juego (fichaje de Sommer). Único cambio: **valor de mercado de Sommer 2.500.000 -> 77.700.000 EUR** dentro del blob, que se recomprimió con miniz (el tramo cambiado; los demás copiados tal cual). Si el juego carga la partida y muestra 77,7 M, la escritura del blob queda validada (necesaria para inscribir fichajes en competiciones).

8 oct 19:45 - Prueba 18 - Ranura 2 (ML00000001, md5 054f2e78...) - El programa ficha a Sommer (Inter -> City) - [PENDIENTE] EN PRUEBA Desde el respaldo de inicio de temporada (4/8/2026). Sommer al City con dorsal 12 (el de su selección: el 1 está ocupado), contrato 3.186.500 EUR/año hasta el 30/6/2027, pago 2.575.000 EUR, Josep Martínez a la portería del Inter. Comparado antes con el fichaje que hizo el juego (§20). Qué mirar: plantilla, alineación, ficha y contrato de Sommer, presupuesto, el Inter; y jugar un partido oficial.

8 oct 19:08 - Resultado prueba 17 - Ranura 1 - Blob reescrito por el programa - [PROBADO] La partida carga normal y la ficha de Sommer muestra **Valor de mercado 77.700.000 EUR** (antes 2.500.000). El presupuesto no cambió (160.665.600 EUR): solo cambió el dato pedido. **El juego acepta la zona comprimida recomprimida por Phoenix Mercado (miniz)**.

8 oct 19:20 - Resultado prueba 18 - Ranura 2 (fichaje de Sommer hecho por el programa) - [OJO] CASI (se corrigió) En pantalla todo bien: Sommer en el City (27/27), sueldo 3.186.500 EUR, contrato hasta 30/6/2027, presupuesto 160.665.600 EUR, ppto. salarial 446.900 EUR, Inter con Josep Martínez de arquero. Fralex lo puso de titular, el partido **arrancó con Sommer** (está bien inscrito); al salir del partido el juego volvió al menú sin guardar, pero antes había **autoguardado** la ranura 2 (13/8). En ese autoguardado: todo sigue igual salvo que **el juego creó su propio contrato** para Sommer (nº 25, cláusula 2.400.000 EUR, calculado desde la ficha del blob) y el nuestro (nº 28) quedó **duplicado** -> la suma de sueldos supera el tope salarial. **Corrección**: el programa ya no escribe el contrato en la tabla I; solo la ficha del blob (el juego hace el resto).

8 oct 19:25 - Prueba 19 - Ranura 2 (ML00000001, md5 1ba38ab8...) - Sommer fichado v2 (sin contrato en la tabla I) - [PENDIENTE] EN PRUEBA

8 oct 19:26 - Resultado prueba 19 - Ranura 2 -> partido simulado -> ranura 3 (ML00000002, md5 432531d3...) - [PROBADO] Fecha 15/8/2026. Sommer sigue en el City (dorsal 12, titular de arquero), **jugó el partido** (tabla de estadísticas C: 1 partido), **un solo contrato** (el que crea el juego, nº 25, 3.186.500 EUR/año, cláusula 2.400.000 EUR), inscrito en las 6 competiciones, presupuesto de fichajes 160.665.600 EUR y **presupuesto salarial positivo (902.100 EUR)**: el juego subió el tope a 147.515.200 EUR). **Fichar PARA el equipo del usuario queda validado en el juego**. Además: el juego relee el archivo al cargar la ranura, así que se puede cambiar un guardado con el juego abierto mientras no esté cargado (Fralex salió y entró a la ranura 2 y vio la versión nueva).

2026-10-09 - Sider - Prueba 1 «buzón» (phoenix.lua) - INSTALADA, ESPERANDO A FRALEX

Instalada en la carpeta del modo ConmeGol Extras\ConmeGOL Patch 26\SiderAddons, porque el switcher copia esa carpeta encima del juego con robocopy:

- modules\phoenix.lua;
- la línea lua.module = "phoenix.lua" (línea 199, la última de los módulos);
- content\phoenix\avisos.txt.
- Respaldo: sider.ini.respaldo-phoenix-20261009. El diff confirma que solo cambió 1 línea.
- Antes de instalar: sintaxis comprobada con LuaJIT y simulación con un contexto falso de Sider (lee el aviso con tildes y ñ).

- Los avisos en vivo se escriben en la raíz: Conmegol Patch\SiderAddons\content\phoenix\avisos.txt, porque sider_dir es la raíz.
- Resultado en el juego: (pendiente)
- **Resultado 1 (00:34): NO cargó.** En sider.log: "init" function returned an error: phoenix.lua:145: attempt to call global 'pcall' (a nil value) -> Module (phoenix.lua) is NOT activated. El juego siguió funcionando normal. **Causa:** el Lua de Sider 7.3.3 **no trae pcall**. El volcado de env.lua en el mismo log lista los globales que sí existen: assert, pairs, ipairs, tostring, tonumber, type, error, unpack, collectgarbage, table, string, math, os, io, fs, ffi, jit, bit, zlib, memory, match, audio, input, log. **Arreglo (v0.2):** sin pcall. Los errores se cuentan con una «bandera» (Sider ya atrapa el error de cada evento) y el módulo se apaga solo tras 5 seguidos. Probado en LuaJIT con un entorno igual al de Sider (sin pcall) y con errores simulados. **Lección:** probar siempre con el entorno restringido de Sider, no con Lua normal.
- **Resultado 2 (00:39): FUNCIONÓ.** Fralex reinició el juego con el switcher (Shift+R no sirve aquí: solo recarga módulos que ya estaban activos). El overlay muestra «PHOENIX EVOLUTION - puente en vivo v0.2-prueba - conectado» con el aviso de bienvenida.
- **Resultado 3 (00:39): TIEMPO REAL CONFIRMADO.** Con el juego abierto se reemplazó avisos.txt en la raíz (tmp + mv, atómico). El overlay mostró el texto nuevo en ~1 s («último aviso 00:39:07» para un archivo escrito a las 00:39:06), sin cerrar el juego. Las tildes, la ñ y los símbolos y -> se ven bien con la fuente Consolas del overlay. El juego siguió estable (pantalla de título).

2026-10-09 - Sider - Prueba 2 «solo mirar» (phoenix.lua v0.3) - INSTALADA, ESPERANDO A FRALEX

Qué hace: con el overlay abierto en PHOENIX EVOLUTION, la tecla **B** busca en la memoria del juego la ficha de Lamine Yamal (ID 162114). Usa dos patrones:

- A = 58 bytes de Player.bin desde el ID (base viva olmosjr23, registro 16179);
- B = el nombre «Lamine Yamal».
- **Cómo busca:** recorre las regiones COMMIT legibles con VirtualQuery (ya declarado por la librería memory de Sider) y salta las zonas con guarda o sin acceso. Revisa 24 MB por cuadro para no congelar el menú. **Solo lee; nunca escribe.**
- **Qué muestra:** por cada coincidencia, la dirección y 5 cualidades decodificadas. El detalle completo va a sider.log (líneas [phoenix] sonda).
- **Prueba previa:** simulación en LuaJIT con memoria falsa (3 regiones, una no legible). Encontró las 2 copias legibles con las cualidades correctas (Vel 90, Ace 93, Reg 93, Fin 81, Pase 82), ignoró la región sin acceso y no tocó nada.
- **Resultado en el juego:** (pendiente)

Resultado 1 (02:41, v0.3): revisó 4407 MB en 3735 regiones en 32,7 s. El juego aguantó.

- Patrón exacto de 58 bytes: **0 coincidencias**. El juego cambia bits al cargar la ficha (forma, lesión...), así que hay que buscar por ID y verificar.
- Nombre «Lamine Yamal»: 6 coincidencias, pero todas eran textos nuestros (el overlay y el log) o zonas que ya habían cambiado.

Resultado 2 (02:43, v0.3): CRASH. Fralex pulsó B otra vez y el juego se cerró.

- dlss5-feed.log: exception 0xC0000005 (reading address 0x6E0000) in sider.dll, justo después de que ReShade recargara sus efectos (02:43:01) y liberara memoria.
- **Causa:** la búsqueda leía DIRECTO zonas de memoria. Se averiguaba qué zonas eran legibles en un cuadro y se leían en los siguientes; otro hilo la liberó en medio -> lectura de memoria inexistente.
- **Daño:** ninguno. Solo se leía. Los guardados (EDIT, ML00000000-2, SYSTEM) tienen fecha anterior al crash.
- **Acción inmediata:** se reinstaló phoenix.lua v0.2 (solo avisos) en la raíz y en la carpeta del modo.

Arreglo v0.4 (NO instalado, espera el OK de Fralex):

- nunca lee la memoria directo: la **copia con ReadProcessMemory** sobre el propio proceso (si la zona ya no existe, devuelve «falló» en vez de cerrar el juego) y busca sobre la copia;
- solo revisa memoria privada (heap) y salta la mapeada (gráficos y ReShade);
- busca el ID (4 bytes) y verifica 5 cualidades (>= 4 iguales);
- además busca «LAMINE YAMAL»;
- quita duplicados.
- **Simulado:** zona «liberada» a mitad de la búsqueda (sin crash, contada como lectura fallida), zona mapeada saltada, 2100 IDs falsos descartados, ficha con un bit de forma cambiado encontrada.
- **Lección:** en el juego, otros hilos (ReShade, DLSS, carga de texturas) liberan memoria en cualquier momento. **Nunca leer memoria ajena directo; siempre copiar con una función que pueda fallar sin cerrar el juego.**

Resultado 3 (02:53, v0.4): FICHA ENCONTRADA, sin crash.

- 2728 MB (solo memoria privada) en 6,6 s, 0 lecturas fallidas, 24 candidatos de ID.
- **1 ficha verificada en 0x7ff4d8f80640**, idéntica a Player.bin (8 bytes en cero + ID): Velocidad 90, Aceleración 93, Regate 93, Finalización 81, Pase raso 82. Foto de Fralex.
- El nombre aparece además en **0x7ff4d932d570**, una segunda ficha cuyos bytes difieren (por ejemplo, el byte 66 es 0x31 en vez de 0x00). Puede ser la copia «viva» que usa el juego.
- Siguiente paso: v0.5 lee y compara las dos copias con el archivo, todavía solo leyendo.

Resultado 4 (02:55, v0.5, Fralex en la pantalla de habilidades de Lamine):

- 2926 MB en 3,8 s, sin crash.
- **Ficha verificada, ahora en 0x55b7380** (el juego la movió desde 0x7ff4d8f80640: las direcciones cambian, siempre hay que buscarla).
- Las «Copias 1-5» **no eran fichas**: eran otras listas con el nombre repetido a la misma distancia. Sus cualidades salían en 40 porque no son registros de jugador. Arreglo: exigir el ID en +8 para llamarla «copia».
- **Calibración**: las 15 cualidades que muestra el juego en esa pantalla coinciden exactamente con nuestro mapa de bits, **incluida Potencia de tiro 78 (bit 358), que pasa de «por confirmar» a CONFIRMADA**.

2026-10-09 - Sider - Prueba 3 «escribir un número» (phoenix.lua v0.6) - INSTALADA CON OK DE FRALEX

- **Teclas**: V = buscar la ficha y poner Velocidad 99 **solo en la memoria**; N = devolver el byte original.
- **Qué cambia: un solo byte**, el 38 de la ficha (Velocidad = bits 306-311 = bits 2-7 de ese byte). Los 2 bits bajos se conservan.
- **Antes de escribir, en el mismo instante**: se vuelve a leer la ficha y se comprueban ID + nombre + >=4 cualidades.
- **Cómo escribe**: con WriteProcessMemory, que falla sin cerrar el juego si la zona ya no existe. Después lee de nuevo para confirmar.
- **N** solo devuelve el byte si todavía tiene el valor escrito por nosotros.
- **cdef con nombres por versión** (phx06_*), porque tras Shift+R el Lua es el mismo y no se puede redeclarar.
- **Simulado**: 2 fichas legibles cambiadas de 90 a 99, la zona «liberada» intacta; N las devolvió a 90.
- **Resultado en el juego**: (pendiente)

Resultado 1 (03:04, v0.6): se escribió Velocidad 99 en 1 copia (0x55b7380), pero la pantalla de habilidades de Lamine (Día de partido, Arsenal-Barça) **siguió mostrando 90**.

- El log muestra **4 copias** de la ficha (0x4c98a98, 0x55b7380, 0x6058ac0, 0x77b88ae8), con los primeros 96 bytes idénticos.
- 3 se saltaron porque **no tienen el nombre en el byte 129**, y v0.6 lo exigía. Esas copias sin nombre son las que crea y usa el juego (aparecen nuevas al entrar a pantallas).
- El aviso «[02:59] hola» llegó **desde la web** por Phoenix Link: **cadena web -> Link -> buzón -> Sider confirmada de punta a punta**.
- **Arreglo v0.7**: para escribir se exige el ID en +8 y los bytes 12-65 idénticos a Player.bin, salvo el de Velocidad y los bytes 54-57 (forma, lesión y pie malo, que el juego varía). Así se escribe en todas las copias reales sin depender del nombre. Simulado OK (incluida una ficha con el bit de forma cambiado).

Resultado 2 (03:11, v0.7): la pantalla de habilidades siguió mostrando **90**. Pero el log demuestra que **todas** las copias empaquetadas ya tienen 99, incluso las que el juego crea nuevas al entrar a menús (por ejemplo, 0x4e15c38). La escritura en la copia «madre» funciona y se propaga.

- **Conclusión**: la pantalla lee **otra representación**, probablemente la ficha «desempaquetada» (una cualidad por byte o por número), creada antes y no regenerada.
- **v0.8 (solo lectura en B)**: además de lo anterior, busca 6 formas desempaquetadas de las cualidades de Lamine (orden de pantalla y orden del archivo; valores tal cual y -40; bytes, u16 y u32), sin la velocidad. Simulado OK.
- **Idea de Fralex**: usar los menús del juego («Datos Actual. en vivo») en vez del overlay de Sider. Se investigó: los «live update» de los parches (por ejemplo, EPP) son bases de datos servidas por Sider al arrancar, no la función nativa del juego.

Resultado 3 (03:17, v0.8): 0 coincidencias en las 6 formas desempaquetadas. La pantalla sigue en 90.

- Hay **7 copias**: las nuevas nacen con 90 aunque las anteriores ya tenían 99, y todas tienen el formato exacto del archivo (8 ceros + ID).
- **Nueva hipótesis**: el juego **vuelve a leer commonletc\pesdb\Player.bin** (vía Sider livecpk) cada vez que necesita al jugador, en vez de tener una copia madre.
- Si se confirma, el camino limpio es servirle **nuestro** Player.bin con livecpk_get_filepath: sin tocar memoria, y

cada pantalla nueva vería los cambios.

- **Técnica vista en el parche:** lib\CommonLib.lua (zlac) usa el evento livecpk_read(ctx, filename, addr, len, total_size, offset) para leer CompetitionEntry.bin y CompetitionRegulation.bin.
- **v0.9 (espía pasivo):** registra livecpk_read y solo cuenta y anota cuándo empieza cada lectura de pesdb*.bin. No lee ni cambia datos. Simulado OK.

2026-10-09 - Espía de lecturas (v0.9) - RESULTADOS y prueba 4 «nuestro libro»

Cuándo lee el juego pesdb\Player.bin (log del espía): al arrancar (03:28:16), en «Datos del sistema: Cargando» (03:28:29) y **al entrar y salir del modo EDITAR (03:33:02 -> lectura #3)**. No lo relee al entrar a la pantalla de habilidades, al cambiar «Datos Actual. en vivo» ni al entrar a la Liga Máster (que lee installversionplayer.bin).

- => **Botón nativo para recargar la base sin reiniciar: entrar y salir de EDITAR.**
- **«Datos Actual. en vivo -> Activar»:** el juego responde «Los servicios en línea de este título finalizaron el 25/08/2022». La actualización en vivo nativa depende del servidor de Konami (cerrado), así que no sirve como canal.

Prueba 4 instalada: nueva raíz livecpk\Phoenix-DB puesta **antes** de olmosjr23\Database en sider.ini (raíz y carpeta del modo; respaldo sider.ini.respaldo-phoenixdb-20261009; diff = 1 línea).

- Sirve un Player.bin propio: base viva de olmosjr23 descomprimida, **1 solo byte cambiado** (Lamine, byte 38 -> Velocidad 99) y recomprimida WESYS (zlib 9, cabecera original con tamaños nuevos; verificada de ida y vuelta).
- En _PhoenixMercado_prueba\ldb\ hay variantes v90, v95 y v99 para la prueba en vivo.
- **Plan:** (a) reiniciar -> la pantalla debe mostrar 99; (b) con el juego abierto, cambiar el archivo a v95 -> entrar y salir de EDITAR -> la pantalla debe mostrar 95.
- **Resultado A (03:41): [PROBADO] EL JUEGO USA NUESTRO PLAYER.BIN.** Tras reiniciar, la pantalla de habilidades de Lamine muestra **Velocidad 99**, servida por la raíz livecpk\Phoenix-DB con 1 byte cambiado. **Cambiar stats «al reiniciar» queda probado.**

Resultado B (03:42-03:46): [HIPÓTESIS] sin confirmar.

- Se cambió el archivo a v95 con el juego abierto (03:41:22). El juego releyó Player.bin al salir de Editar (03:41:57, mismo tamaño).
- Pero en un amistoso nuevo (Inter-Barça) siguió 99 y no apareció ninguna copia con 95.
- **Ensuciado:** se pulsó V (escribir en memoria) y volvió a poner 99 en una copia que tenía 90 («byte c8 -> ec»).

v0.10:

- **V y N desactivadas:** el camino elegido es el archivo, no la memoria.
- **Huella de cada lectura de player.bin:** suma cada 64 bytes del trozo que entrega Sider, con su offset y largo. Así se sabe si el juego leyó v95 o v99.
- Prueba limpia: arrancar con v95 -> debe verse 95. Cambiar a v90 con el juego abierto -> Editar -> amistoso nuevo -> mirar y comparar huellas.

2026-10-09 03:53 - CAMBIO DE STATS EN VIVO, SIN REINICIAR - CONSEGUIDO

- **Huella (v0.10):** al arrancar, el juego leyó **exactamente** el Player.bin v95 (sumas 1785225260 / 304097887). La pantalla mostró **95**.
- **03:51:21:** con el juego abierto se reemplazó el archivo por **v90** (tmp + mv).
- **Solo entrar y salir de EDITAR (03:51:51):** el juego relee Player.bin y la huella es la de **v90** (3958249664 / 1038543739). **Pero el amistoso sigue en 95.** Editar carga la base en un borrador propio y lo descarta al salir.
- **EDITAR -> CARGAR** (cargar EDIT00000000, sin guardar; 03:53:45, huella v90): amistoso nuevo -> **Velocidad 90**. [PROBADO]

MÉTODO PROBADO:

- Phoenix Sync genera Player.bin con los cambios (base viva + bytes cambiados, WESYS/zlib).
- Se coloca de forma atómica en la raíz livecpk\Phoenix-DB, que va antes de olmosjr23\Database.
- El jugador pulsa **Editar -> Cargar** (un botón nativo del juego).
- En el siguiente partido o pantalla nueva se ven los cambios. Sin reiniciar y sin tocar la memoria.
- **Estado al cerrar la prueba:** Phoenix-DB tiene v90, cuyos datos son idénticos a los originales de olmosjr23 (solo recomprimido), en la raíz y en la carpeta del modo. Lamine queda como en el parche.
- **Cuidado futuro:** la raíz Phoenix-DB **tapa** el Player.bin de olmosjr23. Cuando el parche se actualice, hay que regenerar nuestro archivo desde la base nueva (Auditor de parches), o se servirían datos viejos.